Pentest-Report RostamVPN redsocket_pfe 10.2026

Red Team Ltd (Internet Mastering & Company, Inc., DBA Red Team Ltd.) · Audit powered by Claude Mythos 5.1

Report date: 2026-10-14 · Engagement RVPN-01 · Work package WP7 · Classification: Public

Field	Value
Prepared for	RostamVPN project (github.com/RostamVPN)
Prepared by	Red Team Ltd
Audit powered by	Claude Mythos 5.1
Lead analysis	Claude Mythos 5.1, operated and reviewed by Red Team Ltd
Report version	1.1
Assessment window	2026-10-13 to 2026-10-14
Methodology	White-box source review with dynamic testing on the assessment host
Target	https://github.com/RostamVPN/redsocket_pfe, branch main, commit 501b6cf

Version history: 1.0 (2026-10-14) assessment of commit 501b6cf; 1.1 (2026-10-14) re-verification of the remediation commits 07fdca8 and ee2a0c6 with Fix Notes and operator Notes added, counts and narrative updated to the post-remediation state.

Index

- Introduction
- Scope
- Test Methodology and Coverage
- Identified Vulnerabilities
  - RVPN-01-701 WP7: IPv4 header length above the received length panics the forwarder (High)
  - RVPN-01-702 WP7: The fill ring is refilled only after a transmit completion, so a run of dropped frames stalls reception permanently (Medium)
  - RVPN-01-703 WP7: Debug levels record every client address per packet and the documented start command enables them (Medium)
  - RVPN-01-704 WP7: The tunnel to the exit is unauthenticated and the exit trusts any packet whose outer source is an EIP (Medium)
  - RVPN-01-705 WP7: The mandatory exit allowlist is rebuilt in a fail-open sequence, is not persisted and matches device names only (Medium)
  - RVPN-01-706 WP7: Relay protection on the exit is a per-device IPv4 sysctl that host operations reset, with no forward-hook policy and no IPv6 control (Medium)
- Miscellaneous Issues
  - RVPN-01-707 WP7: UDP checksums are updated at the TCP offset and fragments are not recognised, so payload is corrupted while the checksum stays valid (Low)
  - RVPN-01-708 WP7: An unquoted heredoc in the deployment script executes a command substitution that overwrites a file under /etc as root (Low)
  - RVPN-01-709 WP7: Gateway resolution releases RX descriptors to the kernel before reading them back, leaking and duplicating umem chunks (Low)
  - RVPN-01-710 WP7: The gateway MAC is learned once and never refreshed, and the ARP cache lookup ignores the entry state (Low)
  - RVPN-01-711 WP7: The XDP program redirects every queue to one socket and replaces or removes foreign programs (Low)
  - RVPN-01-712 WP7: FFI and buffer hygiene: descriptor layout, unbounded umem slices, ARP headroom, netlink errno sign (Low)

  - RVPN-01-713 WP7: The systemd unit lacks CAP_BPF, defines none of the variables it expands, starts in FoU mode and applies no sandboxing (Low)
  - RVPN-01-714 WP7: The configuration test validates only the map, and the map parser accepts duplicate and semantically invalid rows without warning (Low)
  - RVPN-01-715 WP7: Inner source addresses are forwarded unvalidated, so the exit reflects replies into private and local networks (Low)
  - RVPN-01-716 WP7: Unit-test fixtures use public addresses outside the documentation ranges (Info)
  - RVPN-01-717 WP7: Build and supply-chain hygiene: no minimum Rust version, no CI or provenance, no SBOM, build account directory survives the path remap (Info)
- Conclusions
- Appendix A: Dynamic Test Log
- Appendix B: Dependency Inventory and Advisories
- Appendix C: Standards Conformance Checklist
- Appendix D: Open Questions for the Maintainers

Introduction

redsocket_pfe is the packet front end that RostamVPN places in front of its exit servers. The project README describes it as "an XDP program that takes over a NIC queue, rewrites incoming packets' destination from a cloud-NAT'ed private IP back to the public EIP, GRE-encapsulates them toward a back-end, and transmits them back out the same NIC. The back-end answers clients directly with the EIP as source (direct server return), so the return path skips the cloud." The component is a Rust rewrite of getlantern/pfe: a single-threaded AF_XDP forwarder, the binary `pfe`, that runs on a cloud instance and owns one NIC of that instance, plus a systemd unit for the front end and a root-executed shell script that prepares an exit host with GRE devices, a dummy interface holding the public addresses (EIPs), a set of sysctls and an nftables allowlist. Red Team Ltd established from the README and the code that, contrary to the working title of the work package, the component is not an HTTP-level filter or proxy. It never terminates a connection and never parses beyond the IPv4 header. Its one policy object is the map file, and its one filtering decision is whether the destination address of an arriving frame is listed there. Everything else that protects the exit, in particular the port allowlist, lives in the exit-host configuration produced by the deployment script.

The RostamVPN project commissioned Red Team Ltd in October 2026 to assess the complete RostamVPN system under engagement RVPN-01. The engagement was broken into seven work packages: WP1 redsocket_http exit server, WP2 redsocket_proteus exit server, WP3 Android application and cores, WP4 iOS application and extension, WP5 redsocket_wgtcp client library, WP6 WGTCP kernel module, tools and packaging, and WP7 redsocket_pfe front-end filter. This report covers WP7. The testing team invested two working days, 2026-10-13 and 2026-10-14, on the assessment host (macOS on Apple Silicon with Rust stable, cargo audit 0.22, python3 and pandoc). The project supplied the public repository at commit 501b6cf and an earlier static review, `audit-results.txt` in the repository root. In line with the engagement standard, that review was treated as a list of hypotheses only: every item was re-verified against the tree at 501b6cf, items that still held were carried over with fresh evidence and new identifiers, and its central negative conclusion, that no panic is reachable from frame contents, was found not to hold (RVPN-01-701). Version 1.1 of this report adds the re-verification of two remediation passes delivered during the engagement; the assessment baseline remains commit 501b6cf, and file and line references refer to it unless a commit is named.

Methodologically the assessment was a white-box review with dynamic testing. All 2,354 lines of first-party Rust, the three deployment assets, the manifests and the README were read line by line. The crate was built in the debug and release profiles, its nine unit tests were run, clippy was run on the host and, because the AF_XDP, netlink and /proc code is compiled only for Linux, additionally on the x86_64 and aarch64 Linux targets so that this code was type-checked and linted even though it cannot execute on the assessment host; cargo audit was run against the RustSec database of the same day. Because the data plane requires AF_XDP, Red Team Ltd built a hostile-input harness that includes the component's platform-independent modules (`src/packet.rs`, `src/ipmap.rs`) directly and drove them with malformed, truncated, fragmented and randomised frames (one million random frames plus targeted cases) and with hostile map files. The built binary was driven through its command line and its configuration test. The exit-host script was executed in a dry run against command shims that record and grammar-check every `ip`, `sysctl` and `nft` invocation, and the systemd unit was analysed

with a purpose-written checker. The nftables rule set was reviewed statically against the nft grammar, since nft cannot run on macOS. Kernel-side behaviour (AF_XDP buffer layout, XDP redirect semantics, capability checks, sysctl propagation) was derived from the kernel sources and documentation and is marked as such wherever the report relies on it; each such statement carries the Linux command that would confirm it.

Coverage was complete for the first-party source and the deployment assets. The pure packet logic and the configuration parser were exercised dynamically; the AF_XDP socket setup, the XDP program, the netlink attach, the gateway discovery and the main loop were reviewed statically and compile-checked for Linux but were not executed, and no live front end or exit host was accessed. The vendored `libc` crate was verified for integrity but not reviewed line by line.

Red Team Ltd identified six vulnerabilities (one High, five Medium) and eleven miscellaneous issues (nine Low, two Info) against commit 501b6cf. During the engagement the development team addressed thirteen of them in two remediation passes, commit 07fdca8 and the twelve commits ending at ee2a0c6, and Red Team Ltd re-verified every change with the same harness, binary and dry-run tooling. At ee2a0c6 seven items are fixed in full (RVPN-01-701, 702, 707, 708, 709, 712 and 716), six are fixed in part with the remainder stated in their Fix Notes (RVPN-01-703, 705, 706, 710, 713 and 714), and four are open (RVPN-01-704, 711, 715 and 717). The forwarding core is small and, on its intended path, correct: only mapped destinations are forwarded, the map reload is atomic and fail-safe, the dependency surface is a single vendored crate without advisories, and unsafe code is confined to the AF_XDP module. The most serious issue was a remotely triggerable crash: one frame whose IPv4 header-length field exceeds the bytes actually present panicked the forwarder, which then restarted after two seconds (RVPN-01-701, fixed in 07fdca8). A second availability defect stalled reception permanently once 4,096 non-forwarded frames arrived without an intervening transmission, with the process alive so that nothing restarted it (RVPN-01-702, fixed in 6e41112). The other Medium issues concern the trust boundary between the front end and the exit, which at 501b6cf rested entirely on exit-host configuration that was unauthenticated, not persisted and rebuilt in a fail-open sequence (the fail-open rebuild and the missing forward policy are fixed at ee2a0c6; the tunnel remains unauthenticated and the configuration unpersisted), and a debug flag, present in the README's own start command, that recorded every client address per packet (the command and the address at the first debug level are fixed; the higher levels remain). The Low and Info items cover a UDP checksum defect that corrupted payload bytes while leaving the checksum valid (fixed), an unintended root command execution inside the deployment script's closing note (fixed), buffer accounting errors (fixed), interface takeover semantics (open), unit hardening and configuration validation (both fixed in part), test-fixture hygiene (fixed) and build provenance (open).

The remainder of the report is organised as follows. The Scope section fixes the targets, the threat model and the exclusions. The Test Methodology and Coverage section states, area by area, what was examined and how, and summarises the executed dynamic tests. Identified Vulnerabilities and Miscellaneous Issues present the findings with proof of concept, impact, preconditions, recommendation and classification. The Conclusions walk through every area again, record what held up well, give the verdict and prioritise the recommendations. Appendix A is the complete dynamic test log, Appendix B the dependency inventory, Appendix C the OWASP ASVS 5.0.0 conformance checklist and Appendix D the open questions for the maintainers.

Scope

Target. Repository https://github.com/RostamVPN/redsocket_pfe, branch `main`, commit `501b6cf`, assessed as the local checkout of that commit. Crate `pfe-rs` 0.1.0 (edition 2021, licence Apache-2.0), one binary `pfe`, one dependency `libc = "0.2"` locked at 0.2.190 and vendored under `vendor/` via `.cargo/config.toml` (source replacement plus `--remap-path-prefix` for the macOS and Linux home prefixes). Release profile `opt-level = 3`, `lto = true`, no `panic` or `strip` override. No CI definition, no SBOM and no `rust-version` are present.

File	Lines	Role
src/main.rs	698	CLI, signals, main loop, per-frame handling, stats, hex dump
src/xsk.rs	856	AF_XDP via raw syscalls: umem, rings, XSKMAP, hand-assembled XDP program, netlink attach

File	Lines	Role
src/packet.rs	468	Parsing, destination rewrite with checksum update, FoU and GRE encapsulation, ARP (5 tests)
src/ipmap.rs	233	Map-file parsing and atomic swap store (4 tests)
src/neighbor.rs	99	ifindex, MAC, default gateway and ARP cache from /sys and /proc
deploy/backend-dsr.sh	81	Exit-host configuration run as root
deploy/pfe.service	21	systemd unit for the front end
deploy/map.csv.example	12	Map-file format
README.md	138	Design, packet path, operational statements
Cargo.toml, Cargo.lock, .cargo/config.toml	17, 16, 8	Manifest, lockfile v4, vendoring and path remap
vendor/libc	451 files	libc 0.2.190, 133,623 lines of Rust, integrity-checked only

Remediation commits. Version 1.1 re-verified commit 07fdca8 and the twelve commits ending at ee2a0c6 on the same checkout (HEAD read from the repository's ref files; no git command was run). The file table above describes 501b6cf. At ee2a0c6 the first-party Rust totals 2,555 lines (main.rs 780, xsk.rs 868, packet.rs 547, ipmap.rs 240, neighbor.rs 120), `deploy/backend-dsr.sh` has 112 lines, `deploy/pfe.service` 39 lines, and `deploy/pfe` ↪ `.env.example` is new.

The first-party Rust at 501b6cf totals 2,354 lines with 36 `unsafe` tokens (31 in `xsk.rs`, 5 in `main.rs`) and 28 `cfg` ↪ `(target_os = "linux")` gates in `main.rs`. The modules `xsk.rs` and `neighbor.rs` and the Linux half of `main.rs` compile only for Linux; `packet.rs` and `ipmap.rs` compile everywhere.

Threat model. The adversary is a state-level censor that observes, fingerprints and blocks traffic at national scale, operates hostile networks and performs machine-in-the-middle attacks, seizes devices, installs co-resident malicious apps, impersonates the configuration, bootstrap, relay or CDN infrastructure and compromises distribution; alongside it stands the ordinary application attacker on the Internet. For this component the relevant capabilities are: sending arbitrary frames to the front end's public addresses, including malformed, fragmented and source-spoofed IPv4; spoofing the outer source of tunnel packets toward an exit from a network without BCP 38 filtering; occupying the same layer-2 segment as a front end that is not cloud-hosted; obtaining the journal of a seized or subpoenaed front-end instance; and an operator error or host event on the exit. Outcomes are ranked as deanonymisation of clients, silent failure of a front end or exit, use of the exit as a relay into private networks, fingerprintability of the infrastructure, and loss of service.

Exclusions. The vendored `libc` crate beyond integrity verification; the exit service itself (`redsocket_http`, WP1; `redsocket_proteus`, WP2); the cloud control plane (1:1 NAT, security groups, anti-spoofing, flow logs); execution of the AF_XDP data plane, the XDP program, the netlink attach and the gateway discovery on a Linux host; the IPv6 forward path, which the README states is not implemented; traffic analysis of the cloud-to-exit leg on the wire; and every directory outside the repository.

Test Methodology and Coverage

Red Team Ltd examined the component area by area. For each area the method is stated here, the executed tests are summarised and the complete log with commands and trimmed output is in Appendix A.

Frame parsing and rewrite (`src/packet.rs`, `src/main.rs` `handle_frame`). Every slice index in `parse_eth`, `parse_arp` ↪ , `rewrite_ipv4_dst`, `l4_offset`, the encapsulators and `handle_frame` was traced against the length guards that precede it. The harness then drove `rewrite_ipv4_dst` and the encapsulators with the exact guard semantics of `handle_frame` (only `ip.len()>= 20` is checked, main.rs:502): header-length nibbles 0 to 15 on 46-byte and 1,500-byte packets, IP options, first and non-first fragments, UDP with and without checksum, and 1,000,000 pseudo-random frames of 14 to 1,728 bytes with the Ethertype forced to IPv4 in 80 % and ARP in 10 % of

cases. Result: one panic class (RVPN-01-701), one UDP checksum defect (RVPN-01-707), options handled correctly.

Filter logic and bypasses. The decision path was traced from the XDP program (xsk.rs:712-720) through the RX ring to the map lookup (main.rs:507-515): what is redirected, what is dropped, what is answered (ARP, main.rs:459-499) and what is forwarded regardless of protocol, port and source (main.rs:524-578). Interface and queue matching was examined at the XDP program, the XSKMAP population and the bind call (xsk.rs:518-540). IPv6 handling was examined in the map parser (IPv6 rows accepted) and the forwarder (Ethertype 0x86DD dropped, IPv6 routes unreachable). Fragmentation was examined in `l4_offset` and in the outer header (DF bit clear, packet.rs:244). Spoofing was examined for ARP (startup resolution and runtime replies), for the inner source (never validated) and for the outer tunnel source at the exit (GRE without key). The kernel-side consequences of the redirect for frames on other queues were derived from `net/xdp/xsk.c` and are marked as not exercised.

Trust boundary between the front end and the exit. The GRE and FoU encapsulators (packet.rs:259-287) were compared with the exit-side tunnel configuration (backend-dsr.sh:24), the sysctls (:28-29, :40-46) and the nftables table (:53-58). The script was run in a dry run against PATH shims so that every `modprobe`, `ip`, `sysctl` and `nft` invocation was captured in order, with the shim `nft` enforcing the value ranges and element syntax that real nft enforces (`tools/nft_check.py`, a grammar model written for this assessment). Five cases were run: nominal inputs, an out-of-range port, a doubled space in the EIP list, no ingress interface and no variables. The five nft command lines were then reviewed against the nft grammar (`add table`, `add chain` with hook specification, `add rule` with `iifname`, `ip daddr` set, `tcp dport` set and verdict). Points that only a Linux run can confirm are listed in Appendix A.

Resource bounds and DoS resistance. The umem and ring accounting (fill, RX, TX, completion) was traced through `Xsk::new`, `refill`, the main loop and `handle_frame`; the TX-ring-full and drop paths were followed to their recycling; the per-packet `println!` paths were assessed for their effect on the single-threaded loop; the map parser was timed with 200,000 rows and with a line of 1,000,005 fields; the binary's configuration test was timed with the 200,000-row map (0.18 s, 50 MB resident, 200,002 lines of output).

Logging and privacy. Every `println!` and `eprintln!` in `main.rs` was catalogued by trigger (startup, `-d`, `-dd`, `-ddd`, reload, stats, ARP) and by content (client source address, EIP, exit address, frame bytes). Nothing is written to disk by the program; the unit sends stdout to the journal.

Configuration parsing and secret handling. `ipmap.rs` was reviewed and exercised with duplicate keys, extra fields, IPv6 rows, CRLF, a UTF-8 byte-order mark, out-of-range and negative ports, leading-zero octets, CIDR suffixes, mixed families, a NUL byte, an empty file and comment-only files. The CLI parser was exercised with no arguments, unknown arguments, missing values, a queue number above u32, an empty interface and bogus `--tunnel`, `--mode`, `--gateway-mac` and `--gateway4` values under `--configtest`. The component holds no secrets; the map file names the exit hosts.

systemd unit hardening. `deploy/pfe.service` was analysed with `tools/unit_check.py` (directive inventory, `${VAR}` references against `Environment=`/`EnvironmentFile=`, hardening directives present and absent, capability set) and reviewed against systemd.exec(5) and the kernel's capability requirements for `bpf(2)`.

Deployment scripts run as root. `deploy/backend-dsr.sh` was syntax-checked (`bash -n`), executed in the dry run described above, and read for quoting, ordering, idempotency, fail-open windows and side effects. The dry run exposed an unintended command substitution in the closing heredoc (RVPN-01-708). No Python deployment scripts exist in this repository; python3 was used for the assessment's own checkers.

Dependency hygiene and vendoring. `cargo audit` (RustSec database commit edb58f59, 2026-10-14, 1,329 advisories) reported no vulnerabilities and no warnings for the two crates in the lockfile. The vendored crate's `.cargo-checksum.json` package hash was compared with the `Cargo.lock` checksum (identical) and its upstream VCS revision recorded. The release binary was scanned for build paths (no home-directory path survives; the remapped prefix keeps the build account's directory name).

Memory safety and FFI. All 36 `unsafe` sites were reviewed for the invariant they rely on: ring index masking and acquire/release ordering (xsk.rs:238-345), raw slice construction from umem addresses (:355-367), the netlink message assembly (:754-845), the signal handlers (main.rs:208-224) and the frame handling (:429-590). Clippy with the pedantic and nursery groups was run on the Linux target to surface cast and alignment lints (165, none

security-relevant beyond the pointer-alignment casts noted in RVPN-01-712). The TX descriptor type's padding behaviour was measured with a local copy of the declaration (RVPN-01-712).

Dynamic tests executed. Fifty tests were executed and are logged in Appendix A: A01 to A36 against commit 501b6cf, A37 to A41 re-verifying commit 07fdca8 and A42 to A50 re-verifying commit ee2a0c6; the table lists them in groups.

Group	Tests	Tooling	Result	Findings
Build, test, lint, audit	A01 to A09	cargo 1.91.1, clippy 0.1.91, cargo-audit 0.22.2	all pass, 0 warnings, 0 advisories	717 (hygiene)
Hostile-input harness	A10 to A21	Rust test harness including packet.rs and ipmap.rs by path	21 tests pass as written against the observed behaviour	701, 707, 712, 714
CLI and configtest	A22 to A27	built `pfe` binary, hostile map files, strings	as expected; configtest validates only the map	714, 717
Deployment script dry run	A28 to A34	bash, PATH shims, nft grammar model	fail-open sequence and heredoc execution observed	705, 708
Unit and tree checks	A35 to A36	python3 checker, grep, checksum comparison	hardening gaps, undefined variables, fixture addresses	713, 716
Fix verification at 07fdca8	A37 to A41	rebuild, repository tests, harness re-run, dry run	701 and 708 fixed, 707 fixed for UDP, 703 fixed for the README	701, 703, 707, 708
Fix verification at ee2a0c6	A42 to A50	rebuild, 15 repository tests, harness, CLI, dry run, unit checker, tree scan	702, 707, 709, 712, 716 fixed; 703, 705, 706, 710, 713, 714 fixed in part	702 to 716

No background process was started, nothing was bound to the assigned loopback range 20700 to 20799 because the data plane cannot run on macOS, and no external host was contacted. The brief's HTTP-oriented tests (header injection, PROXY protocol handling, slow clients, connection floods) do not apply: the component has no byte-stream listener and no HTTP or PROXY protocol parser, so their hostile-input equivalents were applied to the frame parser and the map parser instead.

Tools and versions. rustc 1.91.1, cargo 1.91.1, clippy 0.1.91, cargo-audit 0.22.2 with RustSec advisory database commit edb58f5974b8 (2026-10-14), Rust targets aarch64-apple-darwin, x86_64-unknown-linux-gnu and aarch64-unknown-linux-gnu, Python 3.15.0, GNU bash 5.3.3, pandoc 3.8.2.1, macOS 26.6.2 on Apple Silicon. Not available on the assessment host: nft, systemd-analyze, shellcheck, Docker, a Linux kernel.

Standards and specifications applied. OWASP ASVS 5.0.0 (30 May 2025) as the control framework for a server component (Appendix C); OWASP Top 10:2025 was considered and found not applicable because the component exposes no web or API surface; CWE for every finding; CVSS v4.0 (FIRST, November 2023) vectors; NIST SP 800-115 for the test approach; NIST SP 800-218 SSDF v1.1, SLSA v1.0, CycloneDX 1.7 and SPDX 3.0.1 for supply-chain observations; the EU Cyber Resilience Act (Regulation (EU) 2024/2847) for the vulnerability-handling observations in the Conclusions. Protocol references: RFC 791 (IPv4, header length and fragmentation), RFC 826 (ARP), RFC 768 (UDP), RFC 9293 (TCP), RFC 1071 and RFC 1624 (checksum computation and incremental update), RFC 2784 and RFC 2890 (GRE and GRE key extension), RFC 2827 (BCP 38) and RFC 3704 (BCP 84) for source-address validation, RFC 5737 and RFC 3849 for documentation addresses, the Linux kernel documents `Documentation/networking/af_xdp.rst` and `Documentation/networking/ip-sysctl.rst`, the nft(8) manual for the rule grammar and systemd.exec(5) for unit hardening.

Identified Vulnerabilities

RVPN-01-701 WP7: IPv4 header length above the received length panics the forwarder (High)

Fix Note: This issue has been fixed by the development team in commit 07fdca8 and verified by Red Team Ltd to be working as expected. `handle_frame` now drops a frame whose header-length field is below 20 or beyond the bytes present (src/main.rs:502-506 at 07fdca8) and `rewrite_ipv4_dst` guards the same condition; the harness re-run showed no panic for any header-length value and none over 1,000,000 random frames (Appendix A, A37 and A38; re-verified at ee2a0c6, A45).

`handle_frame` accepts an IPv4 frame after a single length check, `ip.len()< IPV4_HDR_LEN` (main.rs:502-506), looks the destination up in the map (main.rs:507-515), slides the packet back into the encapsulation headroom and calls `rewrite_ipv4_dst` (main.rs:544-549). That function reads the header-length nibble (packet.rs:196) and immediately computes `checksum(&ip[..ihl])` (packet.rs:199) without comparing `ihl` with `ip.len()`. Any frame whose declared header length exceeds the bytes actually present therefore trips Rust's slice bounds check and the main thread panics. Because `Cargo.toml` sets no `panic = "abort"`, the panic unwinds, `Xsk::drop` (xsk.rs:651-655) detaches the XDP program, the process exits with status 101, and systemd restarts it two seconds later (`Restart=always`, `RestartSec=2`, pfe.service:10-11). During the gap the kernel stack, having no listener, answers the mapped addresses' traffic with RST. A minimum-size Ethernet frame delivers 46 IPv4 bytes, so every header-length nibble of 12 or more on a short packet is sufficient; `pfe` verifies neither the header checksum nor the total length.

Affected: src/packet.rs:196-199 (`rewrite_ipv4_dst`); src/main.rs:502-549 (`handle_frame`).

Proof of Concept: The harness (Appendix A, A10 to A12 and A15) called `rewrite_ipv4_dst` with the exact guard semantics of `handle_frame`. A 46-byte IPv4 packet with `ip[0] = 0x4f`, protocol TCP, destination `10.0.0.10` and the mapped EIP `203.0.113.10` as the new destination produced:

```
thread 'hostile::t01_ihl_exceeds_length_panics' panicked at src/packet.rs:199:26:
range end index 60 out of range for slice of length 46
```

The threshold test showed no panic for header-length nibbles 5 and 11 on a 46-byte packet and a panic for 12 and 15, and no panic for nibble 15 on a 1,500-byte packet. One million pseudo-random frames of 14 to 1,728 bytes produced 6,399 panics, every one of them in this class, and no other panic. The crafted frame for a Linux reproduction is, after a 14-byte Ethernet header with Ethertype 0x0800:

```
4f 00 00 2e 00 00 00 00 40 06 00 00 c6 33 64 4d 0a 00 00 0a (IHL 15, src 198.51.100.77, dst 10.0.0.10)
followed by 26 bytes of padding to reach the 60-byte minimum frame
```

On a Linux host with a veth pair, sending that frame with scapy (`sendp(Ether(dst=mac)/IP(ihl=15, src ↪ ="198.51.100.77", dst="10.0.0.10", proto=6)/Raw(b"\x00"*26), iface="veth1")`) is expected to show the panic in the journal, exit status 101 and a restart two seconds later; Red Team Ltd did not run this step because AF_XDP is unavailable on the assessment host.

Impact: A single unauthenticated frame from anywhere on the Internet stops the front end; one frame every two seconds keeps it stopped, and every client mapped through that front end's EIPs sees refused connections. The attacker's cost is negligible. The defect is a controlled panic, not a memory-safety violation, so there is no path to code execution. Red Team Ltd could not verify whether the cloud fabric delivers a frame whose header length exceeds its received length: a Linux router on the path drops such packets in `ip_rcv_core`, a typical hardware forwarding plane does not check, and a directly attached front end has no such filter. The rating reflects the unauthenticated remote trigger and the single-packet cost.

Preconditions: An EIP, which is public by design, and delivery of the frame to the front NIC.

Recommendation: Validate the header before any rewrite in `handle_frame` and drop with a counter:

```
let ihl = (ip[0] & 0x0f) as usize * 4;
if ip[0] >> 4 != 4 || ihl < IPV4_HDR_LEN || ihl > ip.len() {
   stats.dropped += 1;
   recycle_now(pool);
   return 0;
}
```

Make `rewrite_ipv4_dst` defensive as well, add the harness cases (nibble 0, 12 and 15 on a 46-byte packet) as unit tests, and consider dropping frames whose total-length field exceeds `ip.len()`.

Classification: CWE-130 · OWASP ASVS 5.0.0 V2.2 Input Validation, V15.3 Defensive Coding · CVSS:4.0/AV:N/AC:L/AT:N/PR:

RVPN-01-702 WP7: The fill ring is refilled only after a transmit completion, so a run of dropped frames stalls reception permanently (Medium)

Fix Note: This issue has been fixed by the development team in commit 6e41112 and verified by Red Team Ltd by inspection: the main loop now returns recycled chunks to the fill ring whenever the pool is non-empty, independently of completions (src/main.rs:374-383 at ee2a0c6), so chunks dropped during one batch are refilled before the next `poll`. The loop runs only on Linux; its behaviour under a sustained stream of dropped frames is among the confirmations listed in Appendix D.

`Xsk::new` submits 4,096 of the 8,192 umem chunks to the fill ring, limited by the fill ring size (xsk.rs:557-561, `RING_SIZE` 4096 at main.rs:247). In the main loop, chunks that the kernel has finished transmitting are harvested from the completion ring into `pool`, and the only call that returns chunks to the fill ring is guarded by that harvest:

```
let avail = xsk.comp.available();
for i in 0..avail {
   pool.push(xsk.comp.addr(i) & !CHUNK_MASK);
}
if avail > 0 {
   xsk.comp.release(avail);
   xsk.refill(&mut pool);
}
```

(main.rs:353-360). Every frame that `handle_frame` drops is pushed into `pool` by `recycle_now` (main.rs:445, used at :453, :464, :470, :497, :504, :512, :520, :580, :585) and stays there until a completion arrives. A transmission happens only for a forwarded IPv4 packet (main.rs:569-578) or an ARP reply for a mapped address (main.rs:483-492). Consequently 4,096 consecutive dropped frames without any transmission in between drain the fill ring. From then on the kernel drops every arriving frame (counted in `rx_fill_ring_empty_descs`, printed as `fillEmpty` ↪), `poll` times out once per second (main.rs:362), the loop returns to the top with `avail == 0` and never refills. The process is alive and still prints its statistics every 60 s, so `Restart=always` never fires; only a manual restart recovers the front end. Dropped classes are: Ethertypes other than IPv4 and ARP (IPv6, VLAN-tagged frames), ARP replies and ARP requests for unmapped addresses, IPv4 to unmapped destinations, and IPv4 whose map row is IPv6. The TX-ring-full drop (main.rs:579-581) implies pending completions and heals itself.

Affected: src/main.rs:353-360 (main loop refill), src/main.rs:445 and the `recycle_now` call sites in `handle_frame`; src/xsk.rs:573-584 (`refill`).

Proof of Concept: Static only: the finding rests on the code excerpt above; the loop runs only on Linux and was not exercised. The Linux check: on the veth setup of RVPN-01-701, send 5,000 IPv6 or VLAN-tagged frames (`sendp(Ether(dst=mac)/IPv6(dst="2001:db8::1"), count=5000)`) while no IPv4 traffic flows, then send IPv4 to the mapped address and observe in the `stats:` line that `rx=` and `fwd=` no longer increase while `fillEmpty=` grows; `ss -x` and the process state show it blocked in `poll`.

Impact: Silent, permanent loss of the front end until an operator restarts it. Monitoring that watches the process or the systemd state sees nothing; only the frozen counters reveal the condition.

Preconditions: A sender able to place at least 4,096 frames that `pfe` drops on the bound NIC during a period without any transmission: an on-link host (any Ethertype), an Internet sender of IPv6 if the ENI carries an IPv6 address, or an in-VPC host sending to an unmapped secondary address. On an IPv4-only cloud ENI without neighbours the only non-forwarded traffic is the hypervisor's own ARP, and the condition is unlikely to arise.

Recommendation: Refill whenever the pool is non-empty, independent of completions, both after the completion harvest and after the RX batch so that dropped chunks are returned before the next `poll`:

```
if avail > 0 { xsk.comp.release(avail); }
if !pool.is_empty() { xsk.refill(&mut pool); }
```

Additionally treat `fillEmpty` growth without RX progress as a fatal condition (exit non-zero so that systemd restarts the service) and add `WatchdogSec=` with `sd_notify` keep-alives to the unit.

Classification: CWE-772 · OWASP ASVS 5.0.0 V15.3 Defensive Coding · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA

RVPN-01-703 WP7: Debug levels record every client address per packet and the documented start command enables them (Medium)

Fix Note: Two parts of this issue have been fixed by the development team and verified by Red Team Ltd: the README start command no longer carries `-d` (commit 07fdca8, README.md:37), and at `-d` the per-packet line now masks the client address to its /24, the full address appearing only from `-dd` on (commit 7197ae2, `client_label` at src/main.rs:711-717 at ee2a0c6, repository test `debug_level_one_masks_the_client_address`, Appendix A, A42). The hex dumps at `-dd` and `-ddd`, the map listing at every start and the per-packet output itself remain as described, so the finding stays open for those parts.

Note: `-d` no longer records full client addresses; operators who used it for per-client diagnostics must use `-dd`, which the README now reserves for short diagnostics.

At debug level 1 the forwarder prints one line per forwarded packet containing the client's source address, the EIP, the packet length and the tunnel endpoints: `println!("fwd: {} -> {} ({} bytes)tunnel {} -> {}:{}", ↪ ipv4_str(&src_ip), ipv4_str(&eip.octets()), inner_len, tsrc, tdst, route.tunnel_port)` (main.rs:528-538). At level 2 the first 64 bytes of the client frame (Ethernet, IPv4 and the whole TCP header) and the first 96 bytes of the encapsulated frame are hex-dumped, and at level 3 the complete frames including the application payload of the :80 service (main.rs:539-542, :565-568, `hex_dump` at :686-698). ARP requesters are printed at level 1 (main.rs:479-481). Independently of the debug level, every start prints the full map including the exit hosts' addresses (main.rs:160-173). The unit routes stdout to the journal with timestamps. `deploy/pfe.service` does not pass `-d`, but the README's front-end start command does: `pfe -i <iface> -m /etc/pfe/map.csv -- ↪ tunnel gre -d # or deploy/pfe.service` (README:37). There is no runtime guard, no time limit, no address truncation and no documentation of the retention consequence.

Affected: src/main.rs:528-542, :565-568, :479-481, :160-173, :686-698; README.md:37, :47.

Proof of Concept: Static only; the data plane does not run on the assessment host. The format string at main.rs:529-537 is the record. The Linux check: start with `-d`, send one TCP SYN to an EIP, then `journalctl -u ↪ pfe | grep 'fwd:'` shows the client address against the EIP with a journal timestamp.

Impact: A per-packet, timestamped record of real client addresses against a VPN endpoint on the component with the weakest physical control, a cloud instance whose journal a provider or a legal process can obtain. The threat model ranks this as deanonymisation material. Red Team Ltd rates it Medium rather than High because the record exists only when an operator starts the binary with the debug flag, the shipped unit does not, and the cloud provider sees the same addresses in flow logs regardless; the finding is about creating a durable record on disk. It becomes High if any production front end runs with `-d` (Appendix D). A secondary effect is availability: a `println!` per packet at line rate stalls the single-threaded loop, so any sender can slow the front end by sending traffic.

Preconditions: An operator starts `pfe` with `-d`, `-dd` or `-ddd`, for example by following README:37; access to the instance's journal.

Recommendation: Remove `-d` from README:37 and state the retention consequence next to the option. Separate per-packet logging from `--debug` into an explicit `--log-packets` that auto-disables after a configurable number of seconds, truncates client addresses to /24 by default, rate-limits output and refuses to run under the unit unless an environment override is set. Cap `-ddd` at the headers. Print the map only under `--configtest`. Document journald `SystemMaxUse=` and `MaxRetentionSec=` for front-end instances.

Classification: CWE-532 · OWASP ASVS 5.0.0 V16.2 General Logging, V14.2 General Data Protection · CVSS:4.0/AV:L/AC:L/AT:P/PR:H/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

RVPN-01-704 WP7: The tunnel to the exit is unauthenticated and the exit trusts any packet whose outer source is an EIP (Medium)

The GRE header the front end writes is `gre[0] = 0; gre[1] = 0; gre[2..4] = 0x0800` (packet.rs:284-286): no key, sequence number or checksum. FoU mode is UDP with the checksum field zero (packet.rs:266-271). On the exit, `ip link add "$dev" type gre local "$GRE_LOCAL" remote "$remote" ttl 64` (backend-dsr.sh:24) creates the tunnel without a key, so the kernel selects it on the outer (destination, source) pair alone, and `remote` is the EIP, which is exactly the outer source the cloud NAT produces for legitimate traffic. A datagram with a forged outer source equal to an EIP is byte-for-byte indistinguishable from the front end's. The exit's own controls bound what an injected inner packet can do while they hold: the input allowlist admits only TCP to an EIP on an allowed port (backend-dsr.sh:56-57) and `forwarding=0` discards non-local destinations (:29). This item also affects WP1, whose server receives the injected packets.

Affected: src/packet.rs:277-287 (`encap_gre_v4`), :259-272 (`encap_fou_v4`); deploy/backend-dsr.sh:24.

Proof of Concept: Static only. The Linux lab check, from a host whose uplink does not enforce BCP 38: `send(IP(src=EIP, dst=EXIT)/GRE()/IP(src=PROBE, dst=EIP)/TCP(dport=80, flags="S"))`; a SYN-ACK from the EIP arriving at PROBE confirms injection. On the exit, `nft add rule inet pfe_dsr input meta nftrace set 1` and `nft monitor trace` show the inner packet entering from the tunnel device. Red Team Ltd did not perform this step: it requires an exit host and a spoofing-capable vantage point.

Impact: With the current script intact: (a) the exit service is reachable while bypassing the front end entirely, including the cloud security groups, flow logs and any filtering added to `pfe` in future; (b) a mapping oracle links EIPs to exits, since an inner SYN to EIP X injected at exit Y with the attacker as inner source yields a SYN-ACK from X only if Y serves X; (c) the full relay and exposure outcomes of RVPN-01-705 and RVPN-01-706 whenever those controls regress; (d) the exit answers GRE from a source that matches no tunnel with ICMP protocol unreachable (kernel `gre_rcv` reject path, derived from `net/ipv4/gre_demux.c`, not exercised), which confirms a GRE endpoint to a scanner. The exit's address is not public by design but is printed into the front-end journal at every start (main.rs:168-173), stored in the map file and recoverable through the oracle.

Preconditions: A vantage point without source-address validation and knowledge of an exit's address.

Recommendation: As a stop-gap add a 32-bit key on both sides: set the K bit and the key in `encap_gre_v4` (header grows to 8 bytes, `GRE_HEADROOM` becomes 42) and `key <n>` on each exit device; document that a key is obscurity against blind injection, not authentication, since an on-path observer reads it. Drop GRE at the exit's physical NIC from any source that is not an EIP (`ip protocol gre ip saddr != { EIPs } drop`), which also suppresses the ICMP reply. As the target design, carry the GRE inside an authenticated tunnel between the front NIC's private address and the exit (ESP transport mode with NAT traversal, or WireGuard) so that the exit accepts only traffic the front end produced.

Classification: CWE-306, CWE-290 · OWASP ASVS 5.0.0 V12.3 General Service to Service Communication Security · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:L/VA:L/SC:N/SI:N/SA:N

RVPN-01-705 WP7: The mandatory exit allowlist is rebuilt in a fail-open sequence, is not persisted and matches device names only (Medium)

Fix Note: The fail-open rebuild and the fallback-device gap have been fixed by the development team in commit 31a497c and verified by Red Team Ltd in the dry run: the script now validates every address and port before acting, loads the table as one `nft -f` transaction that re-creates it in place (deploy/backend-dsr.sh:14-32 and :71-87 at ee2a0c6) and drops the kernel's `gre0`, `gretap0` and `erspan0` devices (:79); every malformed input, including an injection attempt through `EIPS`, was rejected before any command ran and the nominal ruleset passed the grammar model (Appendix A, A49). Persistence of the devices, sysctls and table, and the acceptance of EIP-addressed packets on non-tunnel interfaces, remain open.

The README states "The nftables allowlist is mandatory: DSR exposes the host's entire port space through every EIP" (README:93-94). The script that installs it deletes the existing table first, `nft delete table inet pfe_dsr 2>/dev/null || true` (backend-dsr.sh:53), and then recreates it with four separate `nft` invocations under `set -euo pipefail` (:54-57). If the accept rule at :56 is rejected, the script aborts after the table and a chain with `policy accept` exist and before the drop rule at :57 is installed, and the previous good table is already gone.

Two ordinary inputs trigger this: a port above 65535, and two consecutive spaces in `EIPS` or `ALLOW_TCP_PORTS`, which `tr ' ' ','` (:51-52) turns into an empty set element. Nothing in the tree persists the devices, sysctls or table; the script prints a note (:60-73) and exits. The rules match by device name, `iifname "pfe-gre*"`: the kernel's fallback `gre0` device, if ever brought up, decapsulates GRE from any remote and its packets enter the input hook as `gre0`, which neither rule matches, so they are accepted by policy (derived from `ip_tunnel_lookup` in `net/ipv4/ip_tunnel.c`, not exercised). Finally the EIPs are local /32 addresses on the dummy, so a packet with an EIP destination arriving on any non-tunnel interface is accepted as well. This item also affects WP1.

Affected: deploy/backend-dsr.sh:51-58, :60-73; README.md:84-87, :93-95.

Proof of Concept: The dry run (Appendix A, A30 and A31) executed a copy of the script with only the EUID guard neutralised against shims that log every command and validate `nft` arguments as nft does. With `ALLOW_TCP_PORTS="80 70000"` the recorded sequence ended:

```
17 nft delete table inet pfe_dsr
18 nft add table inet pfe_dsr
19 nft add chain inet pfe_dsr input { type filter hook input priority -10; policy accept; }
20 nft add rule inet pfe_dsr input iifname pfe-gre* ip daddr { 203.0.113.10 } tcp dport { 80,70000 }
    ↪ accept
Error: Value 70000 exceeds valid range 0-65535
exit=1
```

With `EIPS="203.0.113.10 203.0.113.11"` the generated rule contained `ip daddr { 203.0.113.10,,203.0.113.11 }`, which nft rejects as a syntax error, with the same sequence. In both cases no drop rule was installed. Whether real nft accepts the unquoted `pfe-gre*` the shell passes as a wildcard string must be confirmed on Linux; the nft scanner's asterisk-string token suggests it does, and the README uses the same form.

Impact: Every listener bound to 0.0.0.0 on the exit becomes reachable from the Internet through every EIP by the normal front-end path, with no spoofing required, until the operator notices. A reboot without persisted configuration has the same effect, as does `gre0` being up.

Preconditions: An operator input error or a reboot; for the fallback device, `gre0` up.

Recommendation: Validate the inputs before use (addresses parse, ports 1 to 65535), generate one ruleset file and load it with `nft -c -f` followed by `nft -f`, which applies all or nothing and leaves the previous table in place on error:

```
table inet pfe_dsr {
   chain input {
      type filter hook input priority -10; policy accept;
      iifname "lo" accept
      iifname != "pfe-gre*" ip daddr { EIPS } drop
      iifname "pfe-gre*" ip daddr { EIPS } tcp dport { PORTS } accept
      iifname "pfe-gre*" drop
      iifname { "gre0", "gretap0", "erspan0" } drop
   }
}
```

Install the file under `/etc/nftables.d/` with an include from `/etc/nftables.conf`, create the tunnels and the dummy as systemd-networkd `.netdev`/`.network` units ordered before `nftables.service`, persist the sysctls in `sysctl.d` with a udev rule for the per-device values, and keep `gre0` administratively down.

Classification: CWE-636 · OWASP ASVS 5.0.0 V13.1 Configuration Documentation, V13.2 Backend Communication Configuration · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:L/VI:L/VA:L/SC:N/SI:N/SA:N

RVPN-01-706 WP7: Relay protection on the exit is a per-device IPv4 sysctl that host operations reset, with no forward-hook policy and no IPv6 control (Medium)

Fix Note: This issue has been fixed by the development team in commit 31a497c and verified by Red Team Ltd by review of the generated ruleset: the table now carries a forward chain in the `inet` family that drops anything entering or leaving a `pfe-gre*` device (deploy/backend-dsr.sh:81-85 at ee2a0c6), so inner IPv4 and IPv6 are covered independently of the sysctl state (Appendix A, A49). The further hardening in the recommendation

(disabling IPv6 on the tunnel devices, loose rather than disabled reverse-path filtering, re-applying per-device sysctls) was not adopted and stands as a recommendation.

The exit host's only protection against becoming a relay for injected inner packets is `sysctl -qw "net.ipv4.conf.$dev.forwarding=0"` per tunnel device (backend-dsr.sh:29). The nftables table contains an input chain only (:55); no forward chain expresses the intended policy, and no `net.ipv6` setting exists in the script. Two kernel behaviours, derived from source and documentation and not exercised, undermine the sysctl. First, when a write to `net.ipv4.ip_forward` (an alias of `conf/all/forwarding`) changes its value, `devinet_sysctl_forward` calls `inet_forward_change`, which propagates the new value to every interface; `Documentation/networking/ip-sysctl.rst` states for `conf/all/forwarding` that "its change resets all configuration parameters to their default state". A `sysctl --system` at boot that turns forwarding on after the tunnel devices exist, or a container runtime or VPN bring-up that enables forwarding, therefore re-enables forwarding on `pfe-greN`. Second, a `type gre` device delivers inner IPv6 (GRE protocol 0x86DD) as IPv6; the `inet` family drop at :57 covers IPv6 to local destinations, but if IPv6 forwarding is on for any other reason, injected inner IPv6 to non-local destinations is routed, and the input hook never sees forwarded packets. Combined with RVPN-01-704 the exit becomes a spoofable relay into any network it can reach, with the exit's address as the apparent origin. This item also affects WP1.

Affected: deploy/backend-dsr.sh:29, :53-58; README.md:79, :84-87.

Proof of Concept: Static only. The Linux check on an exit: `sysctl -w net.ipv4.ip_forward=0; sysctl -w net.ipv4.ip_forward=1; cat /proc/sys/net/ipv4/conf/pfe-gre0/forwarding` is expected to print `1`, the per-device value having been overwritten; the toggle is needed because writing an unchanged value does not propagate. `sysctl net.ipv6.conf.all.forwarding` shows whether the IPv6 path is open. The injection itself is the lab step of RVPN-01-704 with an inner destination on the exit's LAN or a routable IPv6 address, observed with `tcpdump` on the exit's other interfaces.

Impact: Blind, source-spoofed relay from the Internet into the exit's LAN or any tunnelled private network, attributed to the exit. The relay outcome is ranked second in the threat model; the rating is Medium because it requires the injection of RVPN-01-704 and a host state change.

Preconditions: RVPN-01-704 plus either a change of `net.ipv4.ip_forward` after the tunnel devices were created or IPv6 forwarding enabled on the exit.

Recommendation: Add to the same table `chain forward { type filter hook forward priority -10; policy accept; iifname "pfe-gre*" drop; oifname "pfe-gre*" drop; }`; keep the sysctl as defence in depth; set `net.ipv6.conf.<dev>.disable_ipv6=1` and `net.ipv6.conf.<dev>.forwarding=0` on each tunnel device, since inner IPv6 is unsupported (README:105); re-apply per-device sysctls from a udev rule or a networkd hook so they survive device re-creation and `ip_forward` changes; and prefer `rp_filter=2` (loose) to `0` where the design allows, since `0` on `all` and `default` (:40-41) switches reverse-path validation off for every present and future interface.

Classification: CWE-284, CWE-693 · OWASP ASVS 5.0.0 V13.2 Backend Communication Configuration · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:L/VA:N/SC:L/SI:L/SA:N

Miscellaneous Issues

RVPN-01-707 WP7: UDP checksums are updated at the TCP offset and fragments are not recognised, so payload is corrupted while the checksum stays valid (Low)

Fix Note: This issue has been fixed by the development team in commits 07fdca8 (UDP checksum offset) and ff14dd0 (fragments) and verified by Red Team Ltd to be working as expected: `l4_offset` returns `ihl + 6` for UDP, `ihl + 16` for TCP and `None` for a non-first fragment (src/packet.rs:176-193 at ee2a0c6); the repository tests `rewrite_updates_udp_checksum_at_its_offset` and `rewrite_leaves_fragment_payload_alone` pass, and the harness showed non-first UDP and TCP fragments forwarded with every payload byte intact while the first fragment's checksum is adjusted (Appendix A, A39, A44).

`l4_offset` returns the same offset for both transport protocols, `IPPROTO_TCP | IPPROTO_UDP => ihl + 16` (packet.rs:183-186). The TCP checksum is at offset 16 of the TCP header; the UDP checksum is at offset 6 of the UDP header. For an unfragmented UDP datagram, `rewrite_ipv4_dst` (packet.rs:202-229) therefore leaves the real checksum at `ihl + 6` untouched and applies the RFC 1624 adjustment to payload bytes 8 and 9. Because that adjustment subtracts from the one's-complement sum exactly what the destination change

adds, the datagram still passes UDP checksum verification at the exit with two corrupted payload bytes. The "no checksum" test at :207-209 reads the same payload bytes, and the 0 to 0xffff fix-up at :225-227 applies to them. The fragment-offset field (`ip[6..8] & 0x1fff`) is never consulted, so in a non-first fragment the two bytes at `ihl + 16` are payload for TCP as well and are altered; the reassembled datagram then fails its transport checksum at the exit (TCP and checksummed UDP) or is delivered corrupted (UDP with checksum zero). The repository's own test `rewrite_leaves_udp_none_checksum_alone` (packet.rs:362-376) builds a 28-byte packet, so `ip.len()< l4off + 2` (28 < 38) returns at :203-205 before the UDP path is reached; the test passes without exercising it.

Affected: src/packet.rs:175-187 (`l4_offset`), :202-229 (`rewrite_ipv4_dst`), :362-376 (ineffective test).

Proof of Concept: Harness tests T12 and T04 (Appendix A, A21 and A13). An unfragmented 60-byte UDP datagram with a valid checksum, rewritten from `10.0.0.10` to `203.0.113.10`:

```
T12 UDP checksum field (ihl+6) before [97, e0] after [97, e0]; pseudo-sum after rewrite = 0x0000 (0 means
    ↪ valid)
T12 payload bytes 8..10 before [11, 11] after [df, 0f]
T12 zero-checksum UDP: payload bytes 8..10 after rewrite [df, 0f] (pattern 11 11), checksum field [00,
    ↪ 00]
T04 UDP non-first fragment: payload bytes [16..18] before [11, 11] after [df, 0f]
T04 TCP non-first fragment: payload bytes [16..18] before [11, 11] after [df, 0f]
T04 first TCP fragment (MF=1): payload untouched
```

Impact: Every UDP datagram forwarded to an EIP arrives with bytes 8 and 9 of its payload altered and a checksum that verifies; fragmented TCP toward an EIP is discarded at the exit. Today the exit allowlist admits TCP only (backend-dsr.sh:9, :56), so no production traffic is affected, but any UDP service placed behind the front end (QUIC, WireGuard over UDP, DNS) would fail in a way that checksums do not reveal. There is no attacker leverage against other users.

Preconditions: UDP or fragmented traffic to an EIP.

Recommendation: Return the protocol-specific offset and skip non-first fragments:

```
let frag_off = u16::from_be_bytes([ip[6], ip[7]]) & 0x1fff;
if frag_off != 0 { return None; }
Some(match ip[9] { IPPROTO_TCP => ihl + 16, IPPROTO_UDP => ihl + 6, _ => return None })
```

Replace the 28-byte fixture with datagrams long enough to reach the field and add the T04 and T12 cases as unit tests.

Classification: CWE-682, CWE-20 · OWASP ASVS 5.0.0 V15.3 Defensive Coding · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:

RVPN-01-708 WP7: An unquoted heredoc in the deployment script executes a command substitution that overwrites a file under /etc as root (Low)

Fix Note: This issue has been fixed by the development team in commit 07fdca8 and verified by Red Team Ltd to be working as expected: the backticks at deploy/backend-dsr.sh:71 are escaped, and the dry run of the script at 07fdca8 completed with an empty standard error stream and printed the command text literally (Appendix A, A41); at ee2a0c6 the note refers to `nft list table inet pfe_dsr` (deploy/backend-dsr.sh:101).

The closing note of `backend-dsr.sh` is printed with `cat <<EOF` (backend-dsr.sh:60) and an unquoted delimiter so that `$GRE_LOCAL` and `${DUMMY}` expand (:64, :66-67). Line 71 of the note contains backticks around a command: the text "save via `nft list ruleset > /etc/pfe-dsr.nft` and an include". Bash treats backticks inside an unquoted heredoc as command substitution, so every run of the script executes `nft list ruleset > /etc/pfe-dsr.nft` as root, writes the complete live ruleset of the host (all tables, not only `pfe_dsr`) into `/etc/pfe-dsr.nft` with the default umask, and prints the note without the command text. `set -e` does not act on a failing substitution inside a heredoc. The note then advises an include of that very file; an operator who followed the advice has `/etc/nftables.conf` include a file that is silently replaced by a full ruleset dump at every re-run, so that the next `nft -f /etc/nftables.conf` loads every table a second time. Whether that makes `nftables.service` fail at boot, which would leave the host without any rules, was not verified and is reported as potential. This item also affects WP1.

Affected: deploy/backend-dsr.sh:60, :71.

Proof of Concept: Dry run A29 (Appendix A). The harness ran the script unprivileged, so the redirect of the substituted command failed and bash reported it:

```
backend-dsr.dryrun.sh: line 60: /etc/pfe-dsr.nft: Permission denied
exit=0
```

The printed note lacked the backticked text. As root on an exit the write succeeds silently.

Impact: An unintended root write under `/etc` on every run; disclosure of the complete host ruleset in a world-readable file; potential loss of the host firewall at boot if the printed persistence advice was followed.

Preconditions: Running the script, which is its intended use.

Recommendation: Escape the backticks in the note or replace them with single quotes; alternatively quote the delimiter (`<<'EOF'`) and substitute the two variables with `printf`. Add `shellcheck` to the project's checks; it flags backticks and unquoted heredocs.

Classification: CWE-149 · OWASP ASVS 5.0.0 V13.1 Configuration Documentation · CVSS:4.0/AV:L/AC:L/AT:P/PR:H/UI:N/VC

RVPN-01-709 WP7: Gateway resolution releases RX descriptors to the kernel before reading them back, leaking and duplicating umem chunks (Low)

Fix Note: This issue has been fixed by the development team in commit d2d3d57 and verified by Red Team Ltd by inspection: `resolve_gateway_mac` now collects the chunk addresses of the whole batch while reading it, releases the batch once and refills the collected chunks afterwards (src/main.rs:644-662 at ee2a0c6); the function runs only on Linux and was not executed.

In `resolve_gateway_mac` the success path first publishes the consumed batch to the kernel, `xsk.rx.release(n)` ↪ (main.rs:627), which advances `cached_cons`, and only then reads descriptors for recycling: `let mut junk: ↪ Vec<u64> = (0..n).map(|j| xsk.rx.desc(j).addr & !CHUNK_MASK).collect();` (main.rs:629-630), followed by `xsk ↪ .refill(&mut junk)` (:632). `RxRing::desc` indexes relative to `cached_cons` (xsk.rs:246-250), so after the release it reads the slots of the next, not yet produced, batch. On the freshly mapped ring those slots are zero, so chunk address 0 is submitted to the fill ring while chunk 0 is already there from the initial fill (xsk.rs:557-561), and the chunk that actually held the ARP reply is never returned. The fallback path a few lines later has the correct order (collect at :641, release at :642). The kernel may now hand out chunk 0 for two different packets; in copy mode the frame data is copied at transmit time, so a second packet arriving into chunk 0 while the first is queued for transmission corrupts one of them.

Affected: src/main.rs:625-635 (`resolve_gateway_mac` success path); src/xsk.rs:246-257 (`RxRing::desc`, `release`).

Proof of Concept: Static only; the function runs only on Linux. The code excerpt is the evidence. A Linux check: print `junk` before the refill at main.rs:632 and compare with the descriptor address read before the release; the printed value is expected to be 0 while the consumed chunk is a non-zero multiple of 2,048.

Impact: One leaked chunk per startup batch that contained the reply (typically one) and one duplicated chunk in circulation for the life of the process, which can corrupt one forwarded frame at an unpredictable time. The duplicate is recreated at every restart, so RVPN-01-701 makes it recur.

Preconditions: The gateway MAC is resolved by probe, which happens whenever the ARP cache has no entry and `--gateway-mac` is not given.

Recommendation: Collect the addresses before releasing the batch, as the fallback path does, and factor the two paths into one helper so they cannot diverge.

Classification: CWE-672 · OWASP ASVS 5.0.0 V15.3 Defensive Coding · CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:N/VI:L/VA

RVPN-01-710 WP7: The gateway MAC is learned once and never refreshed, and the ARP cache lookup ignores the entry state (Low)

Fix Note: The cache-lookup part of this issue has been fixed by the development team in commit 99f6196 and verified by Red Team Ltd: `arp_cache_lookup` now requires an entry on the bound interface that is complete (`ATF_COM`) and carries a non-zero unicast address (src/neighbor.rs:72-95 at ee2a0c6); the module's new unit test, which the repository can run only on Linux, was executed on the assessment host by including the module in the harness and passes (Appendix A, A48). The gateway MAC is still resolved once and never refreshed, the counters still count submission rather than delivery, and `--gateway-mac` accepts broadcast and multicast addresses (A43); those parts remain open.

The next-hop MAC for every encapsulated frame is determined once at startup (main.rs:296-320) and never revisited; runtime ARP replies are counted and discarded (main.rs:468-471). The cache lookup `arp_cache_lookup` ↪ (neighbor.rs:85-95) scans `/proc/net/arp` and returns the hardware-address column for the first line whose IP matches, without checking the Flags column (0x2 marks a complete entry) or the Device column. An incomplete or failed entry, which the kernel lists with flags 0x0 and the all-zero hardware address, is accepted as the gateway MAC, as is an entry for the same address on a different interface. The probe path accepts the first reply whose sender protocol address matches (main.rs:625) from any sender. After a gateway MAC change, or a start with the all-zero MAC, every tunnel frame goes to a dead address while `fwd=` and `tx=` keep rising, because they count submission rather than delivery (main.rs:574-576): a silent failure behind healthy counters. On a shared layer-2 segment, which is not the cloud deployment but is possible, a neighbour that answers the probe first receives the entire tunnel leg in cleartext.

Affected: src/neighbor.rs:85-95 (`arp_cache_lookup`); src/main.rs:296-320, :468-471, :594-648, :574-576.

Proof of Concept: Static only. The Linux check: create an incomplete neighbour entry for the gateway (`ip neigh` ↪ `add <gw> dev <nic> nud incomplete` or trigger one with a packet to an unanswering address), start `pfe` without `--gateway-mac`, and observe `gateway mac (arp cache): 00:00:00:00:00:00` followed by rising `fwd=` and nothing at the exit.

Impact: Outage that the counters do not show; on non-cloud segments, interception of the tunnel leg.

Preconditions: A stale or incomplete cache entry at start, or a gateway MAC change during operation.

Recommendation: In `arp_cache_lookup`, require flags with bit 0x2 set and the device equal to the bound interface, and reject all-zero and multicast addresses. Re-resolve the gateway periodically and whenever the completion ring makes no progress for a configurable interval, log MAC changes, and expose the completion count as the delivered metric. In automation pass `--gateway-mac` from infrastructure data.

Classification: CWE-754, CWE-290 · OWASP ASVS 5.0.0 V15.3 Defensive Coding · CVSS:4.0/AV:A/AC:H/AT:P/PR:N/UI:N/VC

RVPN-01-711 WP7: The XDP program redirects every queue to one socket and replaces or removes foreign programs (Low)

The hand-assembled XDP program loads the bound queue number as an immediate, `BpfInsn::new(` ↪ `BPF_MOV64_IMM, 2, 0, 0, queue as i32)` (xsk.rs:716), rather than the frame's `rx_queue_index`, and every one of the 64 XSKMAP slots is pointed at the single socket (xsk.rs:534-539). Every frame from every queue of the netdev is therefore redirected to a socket bound to one queue. In the kernel, `xsk_rcv_check` rejects a frame whose receive queue differs from the socket's queue and the failed redirect drops the frame rather than passing it to the stack (derived from `net/xdp/xsk.c` and the generic XDP path in `net/core/dev.c`, not exercised). On a multi-queue NIC the whole interface goes dark, not "the bound queue" as README:32-34 states; the README's `ethtool -L <iface> combined 1` instruction (README:36) avoids the condition but is not enforced. The attach message carries the mode flag only (xsk.rs:767-804, no `XDP_FLAGS_UPDATE_IF_NOEXIST`), so an XDP program already attached in that mode, for example a DDoS filter, is silently replaced, and the detach at exit (xsk.rs:853-856) removes whatever program is attached without checking that it is the forwarder's. Operationally the generic XDP hook also takes the kernel's own traffic on the NIC (DHCP renewals, neighbour discovery); the consequence for the instance's address lease on the data NIC is raised in Appendix D.

Affected: src/xsk.rs:712-720, :534-539, :767-804, :853-856; README.md:32-34.

Proof of Concept: Static only. The Linux check: `ethtool -L <nic> combined 2`, start `pfe` on queue 0, send flows to the instance's own address that hash to queue 1 (`ethtool -S <nic>` shows the per-queue counters) and observe that the kernel never answers; `ip link show dev <nic>` before and after the start shows the `xdp` program id replaced.

Impact: Lock-out of an instance whose shared NIC is bound; silent removal of third-party XDP protections; the README's description of the blast radius is too narrow.

Preconditions: A multi-queue NIC without the `ethtool -L` step, or a pre-existing XDP program.

Recommendation: Load the key from the context (`BPF_LDX_MEM | BPF_W, r2 = *(u32 *)(r1 + 16)`, the offset of `rx_queue_index` in `struct xdp_md`), populate only slot Q so that other queues fall through to `XDP_PASS`, attach with `XDP_FLAGS_UPDATE_IF_NOEXIST` and fail if a program is present, detach with `XDP_FLAGS_REPLACE` and `IFLA_XDP_EXPECTED_FD`, and correct README:32-34.

Classification: CWE-693 · OWASP ASVS 5.0.0 V13.1 Configuration Documentation · CVSS:4.0/AV:L/AC:L/AT:P/PR:H/UI:N/VC

RVPN-01-712 WP7: FFI and buffer hygiene: descriptor layout, unbounded umem slices, ARP headroom, netlink errno sign (Low)

Fix Note: This issue has been fixed by the development team in commit d089cb3 and verified by Red Team Ltd: the descriptor is `#[repr(C)]` with an explicit `options` field and a compile-time size assertion (src/xsk.rs:224-233 at ee2a0c6), `Umem` bound-checks every `pkt_slice` range (src/xsk.rs:352-368), the ARP reply is built in the received frame (src/main.rs:498-510) and the netlink error is negated before conversion (src/xsk.rs:851); the harness measured the new layout with the `options` word written as zero (Appendix A, A47). Half of the umem is still unused, a nit that remains.

Four hygiene gaps in the unsafe layer were found; none produces a reachable fault with the shipped constants. (a) The TX descriptor is declared `#[derive(Clone, Copy, Debug, Default)] pub struct XdpDesc { pub addr: u64, pub len: u32 }` (xsk.rs:224-228) without `#[repr(C)]` and without the kernel's `options` field, and is written into the 16-byte ring slot with `*p = d` (xsk.rs:279). The kernel validates `options` and rejects descriptors with unexpected bits, counting them as `tx_invalid_descs` (derived from `xp_validate_desc` in `net/xdp/xsk_queue.h`, not exercised). Rust guarantees neither the field order nor the padding content of a default-representation struct. The harness measured a local copy of the declaration (A20): size 16, alignment 8, padding bytes left unwritten (a slot pre-filled with 0xff kept `ff ff ff ff` at bytes 12 to 16); the ring worked because the mapped memory starts zeroed, and a toolchain that copies all 16 bytes would carry stack contents into `options`. (b) `pkt_slice` (xsk.rs:355-358) builds a mutable slice from any address and length without a bound against the umem size; derived addresses are validated only by the one-shot probe (main.rs:376-394), and the ARP reply uses `pkt_addr - FOU_HEADROOM` (42 bytes, main.rs:475) while the probe guarantees only `encap_len`, 38 bytes for GRE (main.rs:378). With the kernel's copy-mode layout (256 bytes of `XDP_PACKET_HEADROOM` plus the 64-byte umem headroom, derived from `net/xdp/xsk_buff_pool.c`, not exercised) the margin is ample, but the invariant is not encoded. (c) `send_nl` passes the negative `nlmsgerr.error` value to `io::Error::from_raw_os_error` (xsk.rs:837-839), so an attach failure prints "Unknown error -1" instead of "Operation not permitted", which matters for diagnosing RVPN-01-713. (d) The pedantic clippy group flags the `*mut u8` casts to `*mut u64` and `*mut XdpDesc` at the ring accessors (xsk.rs:248, :278, :308, :336); the kernel's ring offsets are multiples of 8, so they are aligned in practice, but nothing asserts it. Finally, `NUM_FRAMES` is 8,192 while the fill ring holds 4,096, so half of the 16 MiB umem is never used except for the single probe chunk (main.rs:241-247, :601).

Affected: src/xsk.rs:224-228, :276-281, :355-367, :837-839, :248, :278, :308, :336; src/main.rs:376-394, :475, :241-247.

Proof of Concept: Harness A20 output: `T11 size_of(as shipped)=16 align=8 ; size_of(repr(C)kernel layout) =16` and `T11 ring slot after *p = d (pre-filled 0xff): [88, 77, 66, 55, 44, 33, 22, 11, 42, 00, 00, 00, ff, ff, ff, ff]`. The measurement was taken on the assessment host's toolchain with a copy of the declaration, since `xsk.rs` compiles only for Linux; the result is informational by construction. The remaining items are static.

Impact: Possible silent transmit rejections after a toolchain change; latent out-of-bounds access if the umem constants are changed without revisiting the probe; misleading error text; wasted locked memory.

Preconditions: A toolchain or constant change.

Recommendation: Declare the descriptor `#[repr(C)]` with an `options` field set to 0 and a compile-time size assertion; bound-check `pkt_slice` against the umem length; make the probe require `off >= FOU_HEADROOM.max(encap_len)`; negate the netlink error; add `debug_assert!(p.is_aligned())` in the ring accessors; set `NUM_FRAMES` equal to the ring size.

Classification: CWE-758 · OWASP ASVS 5.0.0 V1.4 Memory, String, and Unmanaged Code · CVSS:4.0/AV:L/AC:H/AT:P/PR:H

RVPN-01-713 WP7: The systemd unit lacks CAP_BPF, defines none of the variables it expands, starts in FoU mode and applies no sandboxing (Low)

Fix Note: This issue has been largely fixed by the development team in commit d136d6b and verified by Red Team Ltd with the unit checker: the unit loads `/etc/pfe/pfe.env` (`deploy/pfe.env.example` is provided), passes `--tunnel gre`, grants `CAP_BPF` and sets fourteen sandboxing directives (deploy/pfe.service:8-33 at ee2a0c6, Appendix A, A50). The service still runs as root without `User=`, keeps `LimitMEMLOCK=infinity`, sets no `RestrictAddressFamilies=` or `SystemCallFilter=`, and the binary's own default remains `fou`; those parts remain open.

Note: The unit now hardcodes `--tunnel gre` and does not start without `/etc/pfe/pfe.env`; FoU deployments must override the unit, and every front end needs the environment file installed before the upgrade.

`deploy/pfe.service` bounds the root service to `CAP_NET_RAW CAP_NET_ADMIN` (pfe.service:14-15). Since Linux 5.8, `bpf(BPF_MAP_CREATE)` and `bpf(BPF_PROG_LOAD)` require `bpf_capable()`, that is CAP_BPF or CAP_SYS_ADMIN, whenever `kernel.unprivileged_bpf_disabled` is non-zero, which is the upstream build default and the distribution default (derived from `kernel/bpf/syscall.c`, not exercised). The forwarder's XSKMAP creation (xsk.rs:680-693) and program load (:712-752) are therefore expected to fail with EPERM under the unit as shipped, and the path of least resistance for an operator is to add CAP_SYS_ADMIN, which negates the bounding set. The `ExecStart` line expands `${PFE_IFACE}`, `${PFE_MODE}` and `${PFE_QUEUE}` (pfe.service:8) but the unit defines no `Environment=` or `EnvironmentFile=` (A35); without a drop-in each expands to an empty argument, `--iface ""` is rejected (main.rs:129-131, confirmed on the host: `error: no interface given (--iface)`), and `Restart=always` with `RestartSec=2` turns that into a restart loop. The unit passes no `--tunnel`, so the binary starts with its default `fou` (main.rs:71) although the README states that only GRE works with a kernel back-end (README:66-69); every forwarded packet then arrives at the exit as UDP to port 5555 with no listener while `fwd=` rises, a silent non-delivery. There is no `User=`, so `AmbientCapabilities=` has no effect, and none of the sandboxing directives are set (A35 lists the 30 absent directives, among them `NoNewPrivileges`, `ProtectSystem`, `ProtectHome`, `PrivateTmp`, `ProtectKernelTunables`, `RestrictAddressFamilies`, `SystemCallFilter`); `LimitMEMLOCK=infinity` is set for a 16 MiB umem.

Affected: deploy/pfe.service:8, :14-18; src/main.rs:71; README.md:37, :66-69.

Proof of Concept: A35 (python3 unit check) and the CLI test with an empty interface (A22). The capability and FoU points are static; the Linux check is `sysctl kernel.unprivileged_bpf_disabled`, `systemctl start pfe` and `journalctl -u pfe` for `BPF_PROG_LOAD failed` or `AF_XDP setup failed`, then `tcpdump -ni <exit-nic> udp port 5555` showing FoU arriving unanswered once variables are supplied.

Impact: The unit as shipped does not start; once patched by hand it forwards into nothing; a compromise of the process has the full reach of root with network capabilities.

Preconditions: Deployment with the shipped unit.

Recommendation: Change the CLI default to `gre` or make `--tunnel` mandatory, and ship the unit as follows (verify that the deployed systemd release knows the `AF_XDP` family name):

```
[Service]
EnvironmentFile=/etc/pfe/pfe.env
ExecStart=/usr/local/bin/pfe --iface ${PFE_IFACE} --mapfile /etc/pfe/map.csv --mode ${PFE_MODE} --queue $
    ↪ {PFE_QUEUE} --tunnel gre --gateway-mac ${PFE_GATEWAY_MAC}
User=pfe
AmbientCapabilities=CAP_NET_RAW CAP_NET_ADMIN CAP_BPF
CapabilityBoundingSet=CAP_NET_RAW CAP_NET_ADMIN CAP_BPF
NoNewPrivileges=yes
```

```
ProtectSystem=strict
ProtectHome=yes
PrivateTmp=yes
ProtectKernelTunables=yes
ProtectKernelModules=yes
ProtectControlGroups=yes
RestrictNamespaces=yes
LockPersonality=yes
MemoryDenyWriteExecute=yes
RestrictAddressFamilies=AF_XDP AF_NETLINK AF_INET
SystemCallArchitectures=native
LimitMEMLOCK=64M
WatchdogSec=30
```

Classification: CWE-250, CWE-1188 · OWASP ASVS 5.0.0 V13.1 Configuration Documentation, V15.2 Security Architecture and Dependencies · CVSS:4.0/AV:L/AC:L/AT:P/PR:H/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N

RVPN-01-714 WP7: The configuration test validates only the map, and the map parser accepts duplicate and semantically invalid rows without warning (Low)

Fix Note: Two parts of this issue have been fixed by the development team and verified by Red Team Ltd on the built binary: the configuration test now rejects invalid `--mode`, `--tunnel`, `--gateway-mac` and `--gateway4` values at parse time (commits 7be6552 and 09d90d7, src/main.rs:80-160 at ee2a0c6), and a duplicate private address is a fatal parse error with its line number (commit ee2a0c6, src/ipmap.rs:136-160; Appendix A, A43 and A46). Semantic checks (EIP equal to the private address, tunnel destinations inside the mapped set, multicast, broadcast and unspecified addresses, port 0, extra fields, IPv6 rows), the reload diff and the unbounded startup listing remain open.

Note: Duplicate map rows are now fatal: a map that previously loaded with later duplicates ignored is rejected at start and at `SIGHUP` reload (the previous map stays in force on reload), so existing map files should be checked with `--configtest` before the upgrade.

`--configtest` returns after the map has been parsed and printed (main.rs:175-178); the XDP mode, the tunnel type, `--gateway-mac` and `--gateway4` are parsed only in `run_linux` (main.rs:253-259, :296-300), so a configuration that would fail at start passes the test. The map parser keeps the first row for a private address and drops later rows silently (ipmap.rs:148-163) and performs no semantic checks: it accepts an EIP equal to the private address, a tunnel destination that is itself a mapped private address (such a row would be re-captured and re-encapsulated by the forwarder), multicast, broadcast and unspecified addresses, port 0, extra trailing fields and IPv6 rows that are counted as routes but never forwarded (README:105). The reload path reports only a route count (main.rs:336-344), and the startup listing is unbounded.

Affected: src/main.rs:175-178, :253-259, :296-300, :160-173, :336-344; src/ipmap.rs:86-163.

Proof of Concept: CLI tests A23 and A24 with the built binary:

```
$ pfe -i lo0 -m good.csv -n --tunnel bogus --mode bogus --gateway-mac zz:zz --gateway4 999.1.1.1 -q
    ↪ 4294967295
pfe-rs starting: iface=lo0 queue=4294967295 mode=bogus map=good.csv routes=1
configtest OK
[exit=0]
$ pfe -i lo0 -m dup.csv -n (two rows for 10.0.0.10)
pfe-rs starting: iface=lo0 queue=0 mode=skb map=dup.csv routes=1
$ pfe -i lo0 -m semantic.csv -n
  route: 10.0.0.10 -> 10.0.0.10 (tunnel 10.0.0.10 -> 198.51.100.20 port 5555)
  route: 0.0.0.0 -> 224.0.0.1 (tunnel 255.255.255.255 -> 239.1.1.1 port 0)
  route: 10.0.0.12 -> 203.0.113.12 (tunnel 10.0.0.12 -> 10.0.0.10 port 5555)
configtest OK
```

Harness T07 and T08 (A16, A17) confirm the parser behaviour in isolation; T09 (A18) confirms that malformed input fails closed, including a UTF-8 byte-order mark, out-of-range ports, leading-zero octets, CIDR suffixes, mixed families and a NUL byte; A26 shows a 200,000-row map producing 200,002 lines of startup output.

Impact: A corrected row appended for an existing private address is ignored while the operator believes the policy updated; a typo in a tunnel destination creates a forwarding loop; a wrong `--tunnel` or `--mode` passes the configuration test and fails at start.

Preconditions: Operator error.

Recommendation: Validate mode, tunnel and gateway arguments in `parse_args`; reject duplicates with the line number; reject non-unicast and unspecified addresses, an EIP equal to its private address, a tunnel destination inside the mapped private set and rows with more than five fields; warn on IPv6 rows; on reload log the rows added, removed and changed; cap the startup listing or print it only under `--configtest`.

Classification: CWE-694, CWE-20 · OWASP ASVS 5.0.0 V2.2 Input Validation · CVSS:4.0/AV:L/AC:L/AT:P/PR:H/UI:N/VC:N/VI

RVPN-01-715 WP7: Inner source addresses are forwarded unvalidated, so the exit reflects replies into private and local networks (Low)

The inner source address is read only for the debug line, `let src_ip: Ipv4Bytes = [ip[12], ip[13], ip[14], ip[15]];` (main.rs:526, used at :531), and forwarding proceeds regardless (main.rs:544-578); the exit script installs no `ip saddr` rule (backend-dsr.sh:56-57). A sender that forges an inner source inside private, loopback, link-local or multicast space, inside the exit's LAN, or equal to an EIP or the exit's own address makes the exit emit a SYN-ACK or RST from the EIP toward that address over its ordinary routing table, since the DSR leg replies directly. The reflection is blind and one-for-one; there is no amplification. Forged sources also appear as clients in the exit's logs.

Affected: src/main.rs:500-582; deploy/backend-dsr.sh:56-57.

Proof of Concept: Static only. The Linux lab check: from a client with reverse-path filtering off, `hping3 -S -p 80 -a 10.0.0.9 <EIP>`, and `tcpdump` on the exit's LAN interface for a SYN-ACK from the EIP to 10.0.0.9.

Impact: Blind reflection into networks the exit can reach, originating from the EIP; log pollution.

Preconditions: A spoofing-capable sender.

Recommendation: In `handle_frame` drop inner IPv4 whose source is in 0.0.0.0/8, 10.0.0.0/8, 100.64.0.0/10, 127.0.0.0/8, 169.254.0.0/16, 172.16.0.0/12, 192.168.0.0/16, 224.0.0.0/4, 240.0.0.0/4 or 255.255.255.255/32, or equals any mapped EIP, private or tunnel address, and count the drops under a new `srcDrop` statistic. At the exit add `iifname "pfe-gre*" ip saddr { the same set } drop` ahead of the accept rule.

Classification: CWE-20, CWE-406 · OWASP ASVS 5.0.0 V2.2 Input Validation · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI

RVPN-01-716 WP7: Unit-test fixtures use public addresses outside the documentation ranges (Info)

Fix Note: This issue has been fixed by the development team in commit c9d4eba and verified by Red Team Ltd: the tree scan at ee2a0c6 found no public address outside the documentation ranges in any source file (66 literals checked; Appendix A, A50).

The engagement's redaction rule requires that the public trees contain no public IPv4 address outside the documentation ranges of RFC 5737. The map-parser tests and the packet tests use globally routable unicast addresses as fixtures: dotted literals at src/ipmap.rs:205, :209, :224 and :230, and byte-array literals at src/packet.rs:338, :341, :367, :373, :388, :401, :424 and :442. Two of them belong to well-known public resolver operators. None is RostamVPN infrastructure, so no live identifier is disclosed, but the values must be replaced before the rule is met. The deployment example, the README and the IPv6 fixtures already use 203.0.113.0/24, 198.51.100.0/24 and 2001:db8::/32 correctly.

Affected: src/ipmap.rs:205, :209, :224, :230; src/packet.rs:338, :341, :367, :373, :388, :401, :424, :442.

Proof of Concept: Tree scan A36 (regular expressions for dotted quads and for four-element byte arrays, classified with Python's `ipaddress` module). The values are deliberately not reproduced here.

Impact: Non-conformance with the publication rule; no security impact. Because two of the values are well known, a reader may also mistake them for infrastructure, which is the confusion the documentation ranges exist to prevent.

Preconditions: None.

Recommendation: Replace every fixture with an address from 192.0.2.0/24, 198.51.100.0/24 or 203.0.113.0/24 (RFC 5737) and add a repository check, run in CI, that rejects other public literals so the condition cannot recur.

Classification: CWE-710 · OWASP ASVS 5.0.0 V15.1 Secure Coding and Architecture Documentation · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:N/SC:N/SI:N/SA:N

RVPN-01-717 WP7: Build and supply-chain hygiene: no minimum Rust version, no CI or provenance, no SBOM, build account directory survives the path remap (Info)

The manifest declares no `rust-version` although the code uses `usize::is_multiple_of` (main.rs:689, xsk.rs:762; stable since Rust 1.87) and `io::Error::other` (xsk.rs:749; 1.74), so older toolchains fail with an unhelpful error, and `Cargo.lock` version 4 requires cargo 1.78 or later. The repository contains no CI definition, no reproducible-build statement, no release signing and no SBOM; NIST SP 800-218 practices PS.3 and PW.4 and SLSA build level 1 (provenance) are therefore not met, and the CycloneDX 1.7 or SPDX 3.0.1 inventory that the EU Cyber Resilience Act's Annex I expects for a product on the EU market does not exist. The release build remaps the macOS and Linux home prefixes to `/build` (.cargo/config.toml:2), which removes the home prefix but keeps the account directory: `strings` on the release binary (A27) shows paths of the form `/build/<account>/.rustup/toolchains/...`, so every shipped binary discloses the build account name; no `strip` or `debug` profile setting is present. On the positive side the dependency surface is one crate, vendored, with the vendored package hash equal to the lockfile checksum, no advisories (A08) and LTO enabled.

Affected: Cargo.toml:1-17; .cargo/config.toml:2; repository root (no CI, SBOM or provenance).

Proof of Concept: A01 to A09 and A27 in Appendix A.

Impact: Reduced reproducibility and auditability; minor information disclosure in the binary.

Preconditions: None.

Recommendation: Add `rust-version = "1.87"`; add a CI job running `cargo build --locked --offline`, `cargo test`, `cargo clippy --all-targets -- -D warnings` for the host and the Linux targets, and `cargo audit`; generate a CycloneDX 1.7 SBOM with `cargo cyclonedx` and SLSA provenance in the release pipeline; build releases in a container with a neutral home path or remap `$HOME` itself; set `strip = "symbols"` in the release profile (panic locations remain in the panic payload).

Classification: CWE-1059, CWE-200 · OWASP ASVS 5.0.0 V15.2 Security Architecture and Dependencies · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:N/SC:N/SI:N/SA:N

Conclusions

Red Team Ltd assessed redsocket_pfe at commit 501b6cf over two days in October 2026 as work package WP7 of engagement RVPN-01. The component is small, candidly documented and built on a narrow idea: a cloud instance gives one NIC to an AF_XDP socket, rewrites the destination of everything that arrives for a mapped private address back to the public EIP, wraps it in GRE toward an exit, and the exit answers the client directly. The assessment confirmed that the idea is implemented as described and that the forwarding core does what the README says on its intended path. It also found that the implementation stops validating its input one field too early, that its buffer accounting has two defects, and that everything which protects the exit lives in a shell script whose failure modes are open. The verdict and the area-by-area account follow.

Forwarding core and frame parsing. The parsing code is disciplined: every index into a frame in `parse_eth`, `parse_arp`, `build_arp_reply`, `l4_offset` and `handle_frame` is preceded by a length check, the Ethertype switch handles exactly two protocols and drops the rest, the checksum arithmetic follows RFC 1624 and was shown to be correct for TCP with and without IP options, and `hex_dump` caps at the frame length. One million randomised frames found exactly one panic class. That class is serious: `rewrite_ipv4_dst` trusts the header-length nibble and slices the header before checking it against the bytes present (RVPN-01-701), so a 60-byte frame from anywhere on the Internet stopped the front end for two seconds, and a trickle of such frames kept it stopped. The earlier static review's statement that no panic is reachable from frame contents is superseded by this

result. The development team fixed the defect in commit 07fdca8 during the engagement, and Red Team Ltd re-verified it: no panic for any header-length value and zero panics over one million random frames. The same function updates the UDP checksum at the TCP offset and does not look at the fragment-offset field (RVPN-01-707); the arithmetic of the incremental update makes the corruption invisible to checksum verification, and the repository's own UDP test never reached the code it was meant to test. The offset was corrected in 07fdca8 with a test that does reach it, the fragment case followed in ff14dd0, and both were re-verified with every payload byte of a non-first fragment intact.

Filter logic and bypasses. The filter is a destination allowlist and nothing more. Red Team Ltd found no path by which an unmapped destination is forwarded, which is the property the design depends on. The converse is also true: a mapped destination is forwarded whatever the protocol, port, fragment status or source (RVPN-01-715), and the README states this openly as the black-hole property. Interface matching is coarser than documented: the XDP program claims every queue of the netdev and the kernel drops, rather than passes, frames from the other queues (RVPN-01-711), so "the bound queue becomes a black hole" should read "the interface becomes a black hole" unless the `ethtool -L` step is enforced. IPv6 is inert on the front end: map rows parse, routes are counted, nothing is forwarded, and the forwarder never claims otherwise; the gap is on the exit, where inner IPv6 injected through the unauthenticated tunnel meets no forwarding control (RVPN-01-706). Spoofing of the outer tunnel source is the central weakness of the design (RVPN-01-704); spoofing of ARP matters only off-cloud and was rated accordingly (RVPN-01-710); the cache lookup was tightened in 99f6196, while the one-time resolution remains.

Trust boundary between the front end and the exit. The front end trusts nothing and the exit trusts the outer source address. The GRE carries no key, the FoU variant no checksum, and the exit's tunnel device is selected on the EIP alone. Every protection that the README calls mandatory therefore lives in `backend-dsr.sh`: the port allowlist, the per-device forwarding switch and the sysctls that make the asymmetric path work at all. The script has improved since the private-tree review that preceded this engagement: the allowlist now matches the device that decapsulated packets actually enter from, the `inet`-family drop also covers inner IPv6 to local addresses, forwarding is switched off per tunnel device, the script is idempotent and refuses to run without its inputs. What remained at 501b6cf was structural. The table was rebuilt with four separate commands after the old one had been deleted, so a port above 65535 or a doubled space in a variable left the exit with an accepting chain and no drop rule (RVPN-01-705); nothing persisted the configuration across a reboot; the forwarding switch was a per-device sysctl that the kernel resets when `ip_forward` changes value and that no forward-hook rule backed up (RVPN-01-706); and the script's closing note, meant to tell the operator how to persist the table, executed a command substitution as root that overwrote a file under `/etc` on every run (RVPN-01-708). The dry run made the first and the last of these visible without a Linux host. Commit 07fdca8 fixed the note and commit 31a497c the other two: the script now validates every address and port before it acts, loads the table as a single transaction that re-creates it in place, drops the kernel's fallback tunnel devices and carries a forward chain; the dry run at ee2a0c6 confirmed that every malformed input, including an injection attempt through `EIPS`, is rejected before any command runs and that the nominal ruleset is grammatically sound. Persistence remains a printed note and the EIPs remain reachable as local addresses on non-tunnel interfaces; the points that only a Linux run can confirm are listed in Appendix D.

Resource bounds and DoS resistance. The forwarder is single-threaded, bounded by one NIC queue, and recycles its buffers correctly on every drop path. The accounting error is in the refill: chunks recycled from dropped frames return to the fill ring only when a transmit completion happens to arrive (RVPN-01-702), so a run of non-forwarded frames with no transmission between them empties the ring and the process waits forever with healthy process state and frozen counters. On an IPv4-only cloud ENI the trigger was unlikely; on a dual-stack ENI or any shared segment it was a single command. The gateway-resolution path released descriptors before reading them back (RVPN-01-709), which leaked one chunk and duplicated another at every start. Commit 6e41112 refills whenever recycled chunks exist and d2d3d57 reorders the gateway path; both were verified by inspection, the loop being Linux-only. The map parser is fast and bounded (200,000 rows in 0.4 s), the TX-ring-full path drops and recycles, and the per-packet debug printing is the only other lever a sender has on the loop.

Logging and privacy. Nothing is written to disk by the program and the periodic statistics are aggregate counters. At the debug levels the picture inverts: one line per forwarded packet with the client's address, and at the higher levels the client's bytes (RVPN-01-703). The README's start command enabled the first level until

commit 07fdca8 removed it, and commit 7197ae2 masks the client address to its /24 at that level; the unit never passed the flag. The full address and the frame bytes remain available at `-dd` and `-ddd`, now documented as short diagnostics, and Red Team Ltd still recommends separating per-packet logging from the debug switch altogether. The startup listing of the map, which names the exit hosts in the front-end journal, belongs behind the configuration test.

Configuration parsing and secret handling. The map parser uses the standard library's strict address parsers, rejects short lines, mixed families, bad ports and empty files, and fails closed on a byte-order mark, a NUL byte, leading zeros and CIDR suffixes. The reload is atomic (`RwLock<Arc<IpMap>>`) and keeps the old map on error; the signal handlers only set atomics. At 501b6cf duplicates vanished silently and the configuration test did not test the arguments that fail at start; both were fixed by ee2a0c6, where a duplicate is a fatal error with its line number and the arguments are validated at parse time. Nonsensical rows are still accepted (RVPN-01-714). The component holds no secrets; the only sensitive configuration datum is the exit address in the map.

systemd unit. At 501b6cf the unit was a starting point rather than a deployment artefact: it could not start as shipped (undefined variables), would not have been allowed to load its BPF program on a current kernel (no CAP_BPF), would have forwarded into nothing if it had (FoU default), and applied none of the sandboxing that a root process with network capabilities warrants. Commit d136d6b addressed all four with an environment file, CAP_BPF, `--tunnel gre` and fourteen sandboxing directives; the service still runs as root with unlimited locked memory and without address-family or system-call filters (RVPN-01-713).

Deployment script run as root. Beyond the three items above, the script is short, uses `set -euo pipefail` ↪ , checks for root and for its inputs, and passes `bash -n`. At 501b6cf it validated none of its inputs before interpolating them into `ip`, `sysctl` and `nft` command lines; since the inputs come from the operator this was not an injection boundary, but it was the mechanism of the fail-open sequence. Since 31a497c every address and port is checked first.

Dependency hygiene and vendoring. This is the strongest area. One dependency, vendored, with the vendored package hash equal to the lockfile checksum, no advisories in the RustSec database of the assessment day, lockfile v4, LTO, and a path remap that keeps the home prefix out of the binary. The gaps are the absence of a minimum Rust version, of any CI, provenance or SBOM, and the account directory that survives the remap (RVPN-01-717), all still open at ee2a0c6; the test fixtures were moved to the documentation ranges in c9d4eba (RVPN-01-716).

Memory safety and FFI. Unsafe code is confined to the AF_XDP module and five sites in the main loop; the ring indices are masked and ordered with acquire and release atomics; the netlink message is assembled from `repr(C)` structures; clippy reports nothing at the default level on the host or on either Linux target. At 501b6cf the descriptor type was the one place where correctness rested on unspecified behaviour (RVPN-01-712), and the padding measurement showed that it worked because the ring memory starts zeroed, not because the type was laid out as the kernel expects; commit d089cb3 made it `repr(C)` with an explicit `options` word, bounded the umem slices and corrected the netlink error sign, and the harness measured the new layout as fully specified.

Standards. Against OWASP ASVS 5.0.0 the component at 501b6cf passed the controls that apply to input handling at the frame level with the two exceptions above (both fixed by ee2a0c6), failed the service-to-service communication controls (no authentication on the tunnel, still open), partially met the configuration controls (documented and, since 31a497c, transactional, but still not persisted), failed the logging control on sensitive data at the debug levels (masked at `-d` since 7197ae2, full at `-dd`), and partially met the secure-architecture and dependency controls (clean dependencies, no provenance). The chapters on web front ends, APIs, sessions, authentication, tokens, OAuth and WebRTC do not apply. Under NIST SP 800-218 the project lacks the build-integrity and provenance practices; under the Cyber Resilience Act, should the service be treated as a product on the EU market, the absence of an SBOM and of a published vulnerability-handling process would need attention before the obligations that apply from December 2027, while the reporting duties that apply from 11 September 2026 concern the project's handling of actively exploited vulnerabilities such as RVPN-01-701 once it is in production.

Verdict. The forwarding design is sound and the code that implements it is careful. The two remediation passes removed every defect that let a remote frame crash or stall the forwarder, corrupted UDP payloads, or left the exit's allowlist open after an input error, and Red Team Ltd re-verified each. What remains is architectural or operational: the tunnel between front end and exit is still unauthenticated (RVPN-01-704), the XDP program

still takes the whole interface (RVPN-01-711), inner sources are still forwarded unvalidated (RVPN-01-715), the exit configuration is still not persisted (RVPN-01-705), the service still runs as root (RVPN-01-713) and the build carries no provenance (RVPN-01-717). With an authenticated tunnel and a persisted exit configuration, Red Team Ltd would consider the front end fit for the threat model it was built for. The exit side needs to be rebuilt as a transactional, persisted, forward-hook-backed ruleset with a keyed tunnel as the first step toward an authenticated one. With those changes, and with per-packet logging taken out of the debug switch, Red Team Ltd would consider the front end fit for the threat model it was built for.

Prioritised recommendations.

1. Add a GRE key now and plan an authenticated tunnel between the front NIC and the exit (RVPN-01-704).
2. Persist the exit configuration (networkd units for the tunnels and the dummy, an nftables include, sysctl.d) and drop EIP-addressed packets that arrive on non-tunnel interfaces (RVPN-01-705); disable IPv6 on the tunnel devices and prefer loose reverse-path filtering (RVPN-01-706).
3. Key the XDP program on the receive queue, attach only if no program is present and detach only the forwarder's own program (RVPN-01-711).
4. Validate inner sources on the front end and at the exit (RVPN-01-715).
5. Separate per-packet logging from the debug switch, cap the dumps and print the map only under the configuration test (RVPN-01-703).
6. Run the service as a dedicated user with ambient CAP_BPF, bound the locked memory and add address-family and system-call filters, and change the binary's default to GRE (RVPN-01-713).
7. Add the semantic map checks, the reload diff and a bound on the startup listing (RVPN-01-714); re-resolve the gateway periodically and reject broadcast and multicast `--gateway-mac` values (RVPN-01-710).
8. Add a minimum Rust version, CI, an SBOM and provenance, and keep the build account out of the binary (RVPN-01-717).

Items fixed and verified at ee2a0c6 (RVPN-01-701, 702, 707, 708, 709, 712 and 716) need only their regression tests kept in place.

Red Team Ltd thanks the RostamVPN maintainers for a repository that states its exposure properties plainly in the README, vendors its single dependency, and keeps its packet logic testable on any platform; those choices made this assessment more thorough than its two days would otherwise have allowed, and the speed and completeness of the two remediation passes made the re-verification straightforward.

Appendix A: Dynamic Test Log

All tests were executed on the assessment host (macOS 26.6.2, Apple Silicon) on 2026-10-14 with the working tree at commit 501b6cf. Scratch work lived under `/tmp/rvpn01/wp7/`: the cargo target directory (`CARGO_TARGET_DIR`), the harness crate, the map files, the dry-run shims and the logs. No background process was started, nothing was bound to the assigned loopback range 20700 to 20799 because the data plane cannot run on macOS, and no external host was contacted. Repository paths are given relative to the repository root. Two assessment artefacts were corrected during the work and are recorded as such: the first version of the nft grammar model mis-tokenised the chain specification and was fixed before the recorded dry runs (A29 to A34), and two harness assertions were rewritten after the first run revealed the UDP offset defect (A13, A21).

A01 Build (debug). `cargo build --offline` in the repository root with `CARGO_TARGET_DIR=/tmp/rvpn01/wp7/target`. Result: pass, exit 0.

```
  Compiling libc v0.2.190
  Compiling pfe-rs v0.1.0 (./redsocket_pfe)
   Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.69s
```

A02 Build (release). `cargo build --offline --release`. Result: pass, exit 0, `Finished release profile [optimized ↪ ] target(s)in 2.24s`; binary 411,792 bytes (debug: 704,120).

A03 Unit tests. `cargo test --offline`. Result: pass, 9 tests.

```
running 9 tests
test ipmap::tests::rejects_mixed_families ... ok
test packet::tests::arp_roundtrip ... ok
```

```
test ipmap::tests::rejects_short_lines ... ok
test ipmap::tests::default_port_and_v6 ... ok
test ipmap::tests::parses_basic_map ... ok
test packet::tests::fou_encap_layout ... ok
test packet::tests::gre_encap_layout ... ok
test packet::tests::rewrite_leaves_udp_none_checksum_alone ... ok
test packet::tests::rewrite_updates_header_and_tcp_checksums ... ok
test result: ok. 9 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

A04 Clippy, host. `cargo clippy --offline --all-targets`. Result: pass, 0 warnings (`Checking pfe-rs v0.1.0`, `Finished`).

A05 Clippy, Linux x86_64. `cargo clippy --offline --all-targets --target x86_64-unknown-linux-gnu`. Result: pass, 0 warnings; this compiles the Linux-only modules `xsk.rs`, `neighbor.rs` and the Linux half of `main.rs`.

A06 Clippy, Linux aarch64. `cargo clippy --offline --all-targets --target aarch64-unknown-linux-gnu`. Result: pass, 0 warnings.

A07 Release check, Linux x86_64. `cargo check --offline --release --target x86_64-unknown-linux-gnu`. Result: pass.

A08 Dependency audit. `cargo audit --no-fetch` and `cargo audit --no-fetch --json` against the local RustSec database (commit edb58f5974b8, 2026-10-14). Result: pass.

```
    Loaded 1329 security advisories (from the local advisory-db)
  Scanning Cargo.lock for vulnerabilities (2 crate dependencies)
{"database":{"advisory-count":1329},"lockfile":{"dependency-count":2},
 "vulnerabilities":{"found":false,"count":0,"list":[]},"warnings":{}}
```

A09 Clippy, pedantic and nursery groups, Linux x86_64. `cargo clippy --offline --all-targets --target x86_64-unknown-linux-gnu -- -W clippy::pedantic -W clippy::nursery`. Result: 165 warnings, informational; the classes with security relevance are the four `*mut u8` to more-strictly-aligned pointer casts (RVPN-01-712). Largest classes: raw-pointer constness casts (19), structure-name repetition (17), `usize` to `u32` truncation casts (14), missing backticks in docs (13), borrow as raw pointer (12), `u64` to `usize` casts (8), could-be-const-fn (7), `usize` to `u16` casts (6).

A10 Harness T01, header length above received length. Harness crate `pfe-harness` including `src/packet.rs` and `src/ipmap.rs` by path; `cargo test --offline -- --nocapture --test-threads=1` with its own target directory. A 46-byte IPv4 packet with `ip[0] = 0x4f`, protocol 6, destination `10.0.0.10`, rewritten to `203.0.113.10`. Result: panic reproduced (finding RVPN-01-701).

```
thread 'hostile::t01_ihl_exceeds_length_panics' panicked at src/packet.rs:199:26:
range end index 60 out of range for slice of length 46
T01 PANIC MESSAGE: range end index 60 out of range for slice of length 46
```

A11 Harness T02, threshold. Nibbles 5, 11, 12, 15 on 46 bytes and 15 on 1,500 bytes. Result: panics for 12 and 15 on 46 bytes only.

```
T02 IHL nibble 0x5 (header 20 bytes, packet 46): panic=false
T02 IHL nibble 0xb (header 44 bytes, packet 46): panic=false
T02 IHL nibble 0xc (header 48 bytes, packet 46): panic=true
T02 IHL nibble 0xf (header 60 bytes, packet 46): panic=true
```

A12 Harness T03, header length zero. Result: the rewritten destination is altered by the checksum fix-up that lands on bytes 16 and 17: `T03 dst after rewrite with IHL=0: [152, 255, 113, 10] (wanted [203, 0, 113, 10])`.

A13 Harness T04, fragments. UDP and TCP non-first fragments (offset 185 and 1), first TCP fragment (MF=1). Result: payload bytes at `ihl+16` altered in non-first fragments, first TCP fragment untouched (RVPN-01-707). The first run of this test also asserted an untouched payload for a first UDP fragment with checksum zero at `ihl+6`; it failed, which exposed the offset defect, and the sub-case was rewritten to TCP with T12 added for UDP.

```
T04 UDP non-first fragment: payload bytes [16..18] before [11, 11] after [df, 0f]
T04 TCP non-first fragment: payload bytes [16..18] before [11, 11] after [df, 0f]
T04 first TCP fragment (MF=1): payload untouched
```

A14 Harness T05, IP options. IHL 6 with NOP NOP NOP EOL, TCP, valid checksums before. Result: pass, both the header and the TCP pseudo-header checksum validate after the rewrite.

A15 Harness T06, randomised frames. 1,000,000 frames of 14 to 1,728 bytes, xorshift seed 0x9E3779B97F4A7C15, Ethertype forced to IPv4 in 80 % and ARP in 10 %, each frame passed through `parse_eth`, `parse_arp` ↪ `/build_arp_reply` or `rewrite_ipv4_dst` plus both encapsulators with the `handle_frame` guard. Result: pass, only the IHL class panics.

```
T06 iterations=1000000 ipv4=799154 arp=100185 other=100661 ihl_overrun_panics=6399 unexpected=0
```

A16 Harness T07, duplicate private address. Two rows for `10.0.0.10`. Result: `routes=1`, first row kept (RVPN-01-714).

A17 Harness T08, semantic validation. Seven cases (EIP equal to private, tunnel destination equal to another row's private address, multicast/broadcast/unspecified with port 0, loopback everywhere, extra trailing fields, IPv6 row, CRLF). Result: all accepted.

A18 Harness T09, rejections. Nine cases (UTF-8 BOM, port 65536, negative port, leading-zero octet, CIDR suffix, mixed family, NUL byte, empty file, comments only). Result: all rejected, fail closed.

A19 Harness T10, large inputs. 200,000 rows and one line with 1,000,005 fields. Result: `routes=200000 parse` ↪ `=409.96ms text=10623584 bytes`; `routes=1 parse=96.48ms`.

A20 Harness T11, descriptor padding. Local copy of the `XdpDesc` declaration from xsk.rs:224-228, assigned into a 16-byte slot pre-filled with 0xff. Result (informational, toolchain-dependent by construction):

```
T11 size_of(as shipped)=16 align=8 ; size_of(repr(C) kernel layout)=16
T11 ring slot after `*p = d` (pre-filled 0xff): [88, 77, 66, 55, 44, 33, 22, 11, 42, 00, 00, 00, ff, ff,
    ↪ ff, ff]
T11 bytes 12..16 (kernel reads these as `options`): [ff, ff, ff, ff]
```

A21 Harness T12, UDP checksum offset. Unfragmented 60-byte UDP datagram with a valid checksum, and a zero-checksum variant. Result: checksum field untouched, datagram still verifies, payload bytes 8 and 9 altered (RVPN-01-707). The first version of the assertion expected verification to fail; the observed value was 0 and the assertion was corrected to document the checksum-preserving corruption. Final run: `test result: ok` ↪ `. 21 passed; 0 failed`.

```
T12 UDP checksum field (ihl+6) before [97, e0] after [97, e0]; pseudo-sum after rewrite = 0x0000 (0 means
    ↪  valid)
T12 payload bytes 8..10 before [11, 11] after [df, 0f]
T12 zero-checksum UDP: payload bytes 8..10 after rewrite [df, 0f] (pattern 11 11), checksum field [00,
    ↪ 00]
```

A22 CLI argument parsing. Debug binary, six invocations. Result: as expected.

```
pfe -> error: no interface given (--iface) [exit=2]
pfe -h -> usage [exit=0]
pfe -i lo0 -m good.csv --unknown -> error: unknown argument '--unknown' [exit=2]
pfe -i lo0 -m good.csv --queue -> error: missing value for --queue [exit=2]
pfe -i lo0 -m good.csv -q 4294967296 -n -> error: invalid queue [exit=2]
pfe -i '' -m good.csv -n -> error: no interface given (--iface) [exit=2]
```

A23 Configuration test with hostile map files. Eight map files under the scratch directory. Result: strict syntax, no semantics (RVPN-01-714).

```
good.csv -> routes=1, configtest OK [exit=0]
dup.csv -> routes=1 for two rows with the same private address [exit=0]
bad.csv -> error: line 1: invalid EIP 'not-an-ip' [exit=2]
empty.csv -> error: no routes found in map file [exit=2]
```

```
/nonexistent -> error: cannot read ...: No such file or directory [exit=2]
bom.csv -> error: line 1: expected at least 4 fields ..., got 1 [exit=2]
v6.csv -> routes=1 (IPv6 row), configtest OK [exit=0]
semantic.csv -> routes=3 (EIP==private, multicast/broadcast/port 0, loop) [exit=0]
```

A24 Configuration test with invalid arguments. `pfe -i lo0 -m good.csv -n --tunnel bogus --mode bogus --gateway-mac zz:zz --gateway4 999.1.1.1 -q 4294967295`. Result: `configtest OK`, exit 0 (RVPN-01-714).

A25 Run without configuration test. `pfe -i lo0 -m good.csv`. Result: map printed, then `pfe-rs packet processing is Linux-only (AF_XDP)`, exit 1.

A26 Large map through the binary. Release binary, 200,000-row map, `--configtest`, measured with `/usr/bin/time -l`. Result: 0.18 s real, 50,855,936 bytes maximum resident set, 200,002 output lines.

A27 Release binary string scan. `strings -a` on the release binary. Result: 0 lines containing the macOS home prefix; remapped toolchain paths of the form `/build/<account>/.rustup/toolchains/...` present (RVPN-01-717).

A28 Script syntax. `bash -n deploy/backend-dsr.sh`. Result: syntax ok.

A29 Dry run, nominal inputs. Copy of `deploy/backend-dsr.sh` with only the EUID guard line replaced (diff of the remainder empty), executed with `env -i PATH=<shims>:/usr/bin:/bin` and `GRE_LOCAL=198.51.100.20 GRE_REMOTES="203.0.113.10 203.0.113.11" EIPS="203.0.113.10 203.0.113.11" ALLOW_TCP_PORTS="80 443" INGRESS_IFACES="eth0"`. The shims log every argument vector; the `nft` shim validates with the grammar model. Result: exit 0, 29 commands, all five `nft` lines valid; stderr shows the heredoc substitution (RVPN-01-708).

```
24 nft delete table inet pfe_dsr
25 nft add table inet pfe_dsr
26 nft add chain inet pfe_dsr input { type filter hook input priority -10; policy accept; }
27 nft add rule inet pfe_dsr input iifname pfe-gre* ip daddr { 203.0.113.10,203.0.113.11 } tcp dport {
    ↪ 80,443 } accept
28 nft add rule inet pfe_dsr input iifname pfe-gre* drop
29 nft list table inet pfe_dsr
stderr: backend-dsr.dryrun.sh: line 60: /etc/pfe-dsr.nft: Permission denied
```

A30 Dry run, port out of range. As A29 with `ALLOW_TCP_PORTS="80 70000"`. Result: exit 1 after 20 commands; table deleted and recreated with an accepting chain, no drop rule (RVPN-01-705).

```
17 nft delete table inet pfe_dsr
18 nft add table inet pfe_dsr
19 nft add chain inet pfe_dsr input { type filter hook input priority -10; policy accept; }
20 nft add rule inet pfe_dsr input iifname pfe-gre* ip daddr { 203.0.113.10 } tcp dport { 80,70000 }
    ↪ accept
Error: Value 70000 exceeds valid range 0-65535
```

A31 Dry run, doubled space in EIPS. As A29 with `EIPS="203.0.113.10 203.0.113.11"`. Result: exit 1 after 27 commands, generated set `{ 203.0.113.10,,203.0.113.11 }`, `Error: syntax error, unexpected comma`, no drop rule (RVPN-01-705).

A32 Dry run, no ingress interface. As A29 without `INGRESS_IFACES`. Result: exit 0, 21 commands, no `accept_local` line, ruleset valid; per README:89-91 the deployment then fails closed.

A33 Dry run, no variables. Result: exit 1, `set GRE_LOCAL=<backend ip> GRE_REMOTES='<frontend eips>' EIPS='<eips>'`, no command executed.

A34 nft grammar review. `tools/nft_check.py --log` over the recorded command lines of A29 to A32 and the model's self-test (valid chain, valid rule, port 70000, double comma). Result: nominal lines valid, the two failure cases rejected with nft's messages. Points the model cannot settle and that the Linux dynamic test must confirm: nft's acceptance of the unquoted `pfe-gre*` wildcard after the shell strips the quotes; the exact rejection text; `nft -f` transactional behaviour for the recommended ruleset; the `gre0` fallback device's acceptance by policy; propagation of an `ip_forward` change to `pfe-greN`; the CAP_BPF requirement under `unprivileged_bpf_disabled`; the drop of frames from other queues; delivery of the RVPN-01-701 frame through the cloud fabric; the RVPN-01-702 stall; and the kernel's treatment of the TX descriptor `options` word.

A35 Unit analysis. `python3 -I tools/unit_check.py deploy/pfe.service`. Result: three `${VAR}` references with no `Environment=`/`EnvironmentFile=`; `AmbientCapabilities=` without `User=`; bounding set without CAP_BPF; 30 hardening directives absent (RVPN-01-713).

A36 Tree scans and integrity. `grep` for IPv4 dotted quads, IPv6 literals and four-element byte arrays in non-vendor files with classification by `ipaddress`; `grep` for mailboxes, URLs, key headers, home-directory prefixes, CDN hostnames; comparison of `vendor/libc/.cargo-checksum.json package` with the `Cargo.lock` checksum. Result: public non-documentation fixture addresses at the twelve locations in RVPN-01-716; no mailbox, key, CDN hostname or absolute path; vendored hash `ce5d3ddc6d3fa000...` equal to the lockfile checksum; upstream VCS revision `7b0ab5528dc7...` recorded.

Fix verification at commit 07fdca8. After the development team's remediation commit the working tree was re-inspected (HEAD at 07fdca8 by reading the ref files; no git command was run) and the following tests were executed.

A37 Rebuild and repository tests at 07fdca8. `cargo build --offline`, `cargo test --offline`, `cargo clippy --offline --all-targets` and `cargo clippy --offline --all-targets --target x86_64-unknown-linux-gnu`, same scratch target directory. Result: pass; 11 tests including the two new ones; 0 clippy warnings.

```
test packet::tests::rewrite_ignores_a_header_length_beyond_the_packet ... ok
test packet::tests::rewrite_updates_udp_checksum_at_its_offset ... ok
test result: ok. 11 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

A38 Harness P01 and P02, header length. The harness, which includes `src/packet.rs` by path and therefore picked up the remediated file, was extended with a `postfix` module whose expectations are inverted where the fix changes behaviour; only that module was run (`cargo test --offline -- postfix --nocapture --test-threads=1`). Result: no panic for header-length nibbles 0, 5, 11, 12 and 15 on 46 bytes (0, 12 and 15 leave the packet untouched, 5 and 11 rewrite it), and zero panics over 1,000,000 random frames both with the new `handle_frame` guard and when calling `rewrite_ipv4_dst` unguarded (RVPN-01-701 fixed).

```
P01 IHL nibble 0xf on 46 bytes: panic=false dst_after=[10, 0, 0, 10]
P02 iterations=1000000 panics=0
```

A39 Harness P03, UDP checksum. Same fixtures as T12. Result: checksum field updated in place, datagram verifies, payload untouched; zero-checksum datagram left alone (UDP part of RVPN-01-707 fixed).

```
P03 checksum field before [97, e0] after [65, df]; pseudo-sum after = 0x0000; payload unchanged=true
P03 zero-checksum UDP: field after [00, 00], payload unchanged=true
```

A40 Harness P04, fragments. Non-first UDP and TCP fragments as in T04. Result: payload bytes at the protocol's checksum offset are still altered (UDP offsets 6 and 7, TCP offsets 16 and 17 of the fragment payload); the fragment part of RVPN-01-707 remains open.

A41 Dry run at 07fdca8. Shims re-created from the remediated script (copy identical apart from the EUID line), nominal inputs as A29. Result: exit 0, standard error empty, 22 commands, six `nft` lines all valid, and the printed note contains the text `nft list ruleset > /etc/pfe-dsr.nft` literally instead of executing it (RVPN-01-708 fixed).

Re-verification at commit ee2a0c6. After the second remediation pass (twelve commits on top of 07fdca8, HEAD ee2a0c6 read from the ref files) the following tests were executed.

A42 Rebuild, tests, lint and audit at ee2a0c6. `cargo build --offline` (debug and release), `cargo test --offline`, `cargo clippy --offline --all-targets` on the host and on both Linux targets, `cargo audit --no-fetch`. Result: pass; 15 tests including the new `rewrite_leaves_fragment_payload_alone`, `rejects_duplicate_private_address`, `parse_args_rejects_bogus_mode_tunnel_and_gateway_values` and `debug_level_one_masks_the_client_address`; 0 clippy warnings; 0 advisories.

```
test result: ok. 15 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

A43 CLI at ee2a0c6. Same map files as A22 to A24. Result: invalid `--tunnel`, `--mode`, `--gateway-mac` and `--gateway4` values are rejected with exit 2; a valid combination with `-q 7 --mode driver --tunnel gre` passes; `dup.csv` fails with `line 2: duplicate private address 10.0.0.10`; `semantic.csv` (EIP equal to private, multicast and

unspecified addresses, port 0, a loop) still passes with `routes=3`; the IPv6 row still passes; the byte-order-mark and empty files still fail closed; `--gateway-mac ff:ff:ff:ff:ff:ff` and a multicast MAC are accepted; the 200,000-row map takes 0.16 s and 35.6 MB and still prints 200,002 lines; the release binary contains no home-prefix path.

A44 Harness Q01, fragments. Non-first UDP and TCP fragments (offsets 185 and 1), a UDP fragment with both MF and a non-zero offset, and a first TCP fragment (MF=1, offset 0). Result: destination rewritten and no payload byte altered in the non-first fragments; the first fragment's TCP checksum adjusted and its payload untouched (RVPN-01-707 fixed).

```
Q01 UDP non-first: dst=[203, 0, 113, 10] payload offsets altered=[]
Q01 TCP non-first: dst=[203, 0, 113, 10] payload offsets altered=[]
Q01 first TCP fragment (MF=1, offset 0): checksum bytes [df, 0f] (adjusted), payload [11, 11, 11, 11]
```

A45 Harness Q02 and P01 to P04 re-run, regressions. Header-length nibbles 0, 12 and 15 leave the packet untouched without a panic; 1,000,000 random frames produce 0 panics; an unfragmented UDP datagram has its checksum field updated and its payload intact. Result: pass (RVPN-01-701 and the UDP part of RVPN-01-707 hold at ee2a0c6).

A46 Harness Q03, map file. Result: a duplicate private address is rejected (`line 2: duplicate private address 10.0.0.10`); EIP equal to private, a tunnel destination inside the mapped set, multicast, broadcast and unspecified addresses with port 0, extra trailing fields and IPv6 rows are still accepted; the byte-order mark, port 65536 and the empty file are still rejected.

A47 Harness Q04, descriptor layout. Copy of the declaration at src/xsk.rs:225-231 (`#[repr(C)]`, `addr`, `len`, `options`), assigned into a 16-byte slot pre-filled with 0xff. Result: size 16, alignment 8, all 16 bytes written, `options` word zero (RVPN-01-712).

```
Q04 size=16 align=8 slot after write: [88, 77, 66, 55, 44, 33, 22, 11, 42, 00, 00, 00, 00, 00, 00, 00]
```

A48 neighbor.rs unit test by inclusion. `src/neighbor.rs` at ee2a0c6 imports `crate::parse_mac`; the harness provides a copy of that function at its crate root and includes the module by path, which the repository compiles only for Linux. Result: `neighbor::tests::arp_table_lookup_wants_a_complete_unicast_entry_on_the_interface` passes on the assessment host (incomplete entry rejected, wrong interface rejected, multicast address rejected, complete unicast entry on the interface accepted) (RVPN-01-710, cache-lookup part).

A49 Dry run of deploy/backend-dsr.sh at ee2a0c6. Shims re-created from the remediated script (copy identical apart from the EUID line, `bash -n` clean); the `nft` shim stores the ruleset passed on standard input to `nft -f -` and validates it with the grammar model's file mode (table block, chains, hook specifications, rules, sets). Eight cases. Result: nominal inputs produce exactly one `nft -f -` invocation followed by `nft list table`, the ruleset below passes the model, and no `delete table` runs outside the transaction; `ALLOW_TCP_PORTS="80 70000"` stops with `invalid TCP port '70000'` before any command; a doubled space in `EIPS` produces a clean set; `203.0.113.999` and a leading-zero octet stop with `invalid IPv4 address`; port 0 stops with `invalid TCP port '0'`; an `EIPS` value containing `} accept; iifname "x" {` stops with `invalid IPv4 address '}'`; no ingress interface still produces a valid ruleset (RVPN-01-705, RVPN-01-706). The `--log` mode of the model does not interpret `-f -` and reports it as unexpected; the file mode is the authoritative check for that invocation.

```
add table inet pfe_dsr
delete table inet pfe_dsr
table inet pfe_dsr {
  chain input {
    type filter hook input priority -10; policy accept;
    iifname "pfe-gre*" ip daddr { 203.0.113.10,203.0.113.11 } tcp dport { 80,443 } accept
    iifname "pfe-gre*" drop
    iifname { "gre0", "gretap0", "erspan0" } drop
  }
  chain forward {
    type filter hook forward priority -10; policy accept;
    iifname "pfe-gre*" drop
    oifname "pfe-gre*" drop
  }
}
```

A50 Unit, fixtures and tree at ee2a0c6. `tools/unit_check.py deploy/pfe.service`: `EnvironmentFile=/etc/pfe/pfe.env` present, `AmbientCapabilities=` and `CapabilityBoundingSet=` include CAP_BPF, fourteen sandboxing directives present, `User=`, `RestrictAddressFamilies=`, `SystemCallFilter=`, `ProtectKernelLogs=`, `ProtectProc=` and `UMask=` absent, `LimitMEMLOCK=infinity` (RVPN-01-713). Fixture scan over `src/*.rs`: 66 address literals, none public outside the documentation ranges, one deliberately invalid literal in a negative test (RVPN-01-716). `grep` for mailboxes, key headers, home prefixes and CDN hostnames: none. `Cargo.toml` still without `rust-version`; no CI definition or SBOM in the tree (RVPN-01-717).

Appendix B: Dependency Inventory and Advisories

Component	Version	Source and integrity	Advisories
libc	0.2.190	Vendored under `vendor/libc` via `.cargo/config.toml`; `.cargo-checksum.json` package hash equals the Cargo.lock checksum (`ce5d3ddc...`); upstream revision `7b0ab552...`; 451 files, 133,623 lines	None (RustSec DB commit edb58f59, 2026-10-14, 1,329 advisories); no yanked warning reported in offline mode
pfe-rs	0.1.0	First-party, edition 2021, Apache-2.0	Findings in this report
Rust toolchain	1.91.1 (assessment host)	rustup stable; no `rust-version` declared in the manifest	Not assessed
Linux kernel ABI	AF_XDP, bpf(2), rtnetlink	Constants hand-coded in xsk.rs:13-45; no uapi binding crate	Not assessed

Build configuration: `[profile.release] opt-level = 3, lto = true`; no `strip`, `debug` or `panic` setting; `Cargo.lock` version 4; `.cargo/config.toml` replaces crates.io with the vendored directory and remaps the macOS and Linux home prefixes to `/build`. No CI definition, no SBOM (CycloneDX 1.7 or SPDX 3.0.1), no provenance (SLSA) and no release signing material exist in the tree. The program opens no outbound sockets other than the AF_XDP socket, a NETLINK_ROUTE socket for the attach and detach, and a connect-only UDP socket used for route selection (neighbor.rs:72-82); it does not contact any service.

Appendix C: Standards Conformance Checklist

OWASP ASVS 5.0.0 (30 May 2025), assessed at section level for a server-side network component with no web, API, session or authentication surface. Results are as assessed at commit 501b6cf; the notes record the status at ee2a0c6. Pass means every applicable control in the section was verified; Partial means at least one applicable control fails; Fail means the section's core control fails; Not applicable means no control in the section applies to this component; Not assessed means the section applies but could not be verified in this work package.

Control	Result	Note
V1.1 Encoding and Sanitization Architecture	Not applicable	No output encoding; binary frames in and out
V1.2 Injection Prevention	Pass	No interpreter, no shell from untrusted input; script inputs are operator-supplied

Control	Result	Note
V1.3 Sanitization	Not applicable	No HTML, SQL, LDAP or template surface
V1.4 Memory, String, and Unmanaged Code	Partial	Unsafe confined and reviewed; descriptor layout and unbounded `pkt_slice` (RVPN-01-712, fixed in d089cb3)
V1.5 Safe Deserialization	Not applicable	No serialised objects; the map is a typed CSV
V2.1 Validation and Business Logic Documentation	Partial	README documents the policy model; no validation rules for the map are written down
V2.2 Input Validation	Fail	Header length unchecked (RVPN-01-701, fixed in 07fdca8); fragments and UDP (RVPN-01-707, fixed in 07fdca8 and ff14dd0); sources (RVPN-01-715, open); map semantics (RVPN-01-714, duplicates and CLI fixed by ee2a0c6, semantics open)
V2.3 Business Logic Security	Pass	Only mapped destinations forwarded; reload atomic and fail-safe
V2.4 Anti-automation	Not applicable	No user-facing operations
V3 Web Frontend Security	Not applicable	No browser surface
V4 API and Web Service	Not applicable	No HTTP, GraphQL or WebSocket surface
V5.1 File Handling Documentation	Partial	Map file format documented; permissions and ownership not
V5.2 File Upload and Content	Not applicable	No uploads
V5.3 File Storage	Pass	Nothing written by the program; the script's unintended write is reported under V13
V5.4 File Download	Not applicable	
V6 Authentication	Not applicable	No users; the tunnel's missing authentication is assessed under V12.3
V7 Session Management	Not applicable	
V8 Authorization	Not applicable	The map is the only policy; no principals
V9 Self-contained Tokens	Not applicable	
V10 OAuth and OIDC	Not applicable	
V11.1 Cryptographic Inventory and Documentation	Fail	No cryptography on the tunnel leg and no documented decision (RVPN-01-704)
V11.2 to V11.7 Cryptography	Not applicable	No cryptographic implementation in the component
V12.1 General TLS Security Guidance	Not applicable	No TLS
V12.2 HTTPS Communication with External Facing Services	Not applicable	
V12.3 General Service to Service Communication Security	Fail	Front end to exit unauthenticated and unencrypted (RVPN-01-704)

Control	Result	Note
V13.1 Configuration Documentation	Partial	Exposure properties documented; persistence (RVPN-01-705, open), fallback device (fixed in 31a497c), interface takeover (RVPN-01-711, open), unit requirements (RVPN-01-713, environment file documented in d136d6b)
V13.2 Backend Communication Configuration	Fail	Fail-open rebuild, no forward policy, sysctl reset (RVPN-01-705 transactional and RVPN-01-706 forward chain fixed in 31a497c; persistence open)
V13.3 Secret Management	Pass	No secrets in the component; none in the tree
V13.4 Unintended Information Leakage	Partial	Build account directory in the binary; exit addresses in the journal at start (RVPN-01-717, RVPN-01-703)
V14.1 Data Protection Documentation	Fail	No statement on client-address handling or journal retention (RVPN-01-703; README documents the levels since 7197ae2, retention still undocumented)
V14.2 General Data Protection	Partial	Nothing stored by default; debug levels record client data (RVPN-01-703; masked at `-d` since 7197ae2, full from `-dd`)
V14.3 Client-side Data Protection	Not applicable	
V15.1 Secure Coding and Architecture Documentation	Partial	Design documented; fixtures (RVPN-01-716, fixed in c9d4eba) and coding rules
V15.2 Security Architecture and Dependencies	Partial	One vendored, audited dependency; no MSRV, CI, SBOM or provenance (RVPN-01-717)
V15.3 Defensive Coding	Partial	Bounds checks throughout except RVPN-01-701 (fixed); accounting defects (RVPN-01-702 and RVPN-01-709 fixed in 6e41112 and d2d3d57; RVPN-01-710 lookup fixed in 99f6196, refresh open)
V15.4 Safe Concurrency	Pass	Single thread; signal handlers set atomics only; `RwLock<Arc<IpMap>>` swap
V16.1 Logging Documentation	Fail	Debug output and its retention consequence undocumented (RVPN-01-703)
V16.2 General Logging	Fail	Client addresses and frame bytes at debug levels (RVPN-01-703; /24 at `-d` since 7197ae2, full address and bytes from `-dd`)

Control	Result	Note
V16.3 Security Events	Partial	Reloads and aggregate drops logged; no reason codes, no gateway change events (RVPN-01-710)
V16.4 Log Protection	Not assessed	Journal configuration is outside the tree
V16.5 Error Handling	Partial	Errors fail closed; panic on malformed input (RVPN-01-701, fixed in 07fdca8); netlink error text wrong (RVPN-01-712, fixed in d089cb3)
V17 WebRTC	Not applicable	

Appendix D: Open Questions for the Maintainers

1. Does any production front end run `pfe` with `-d`, `-dd` or `-ddd`, and what are the journald retention settings on front-end instances? (RVPN-01-703; the severity rises to High if the answer is yes.)
2. Does the cloud fabric in front of the production front ends deliver an IPv4 frame whose header length exceeds its received length, or is it discarded before the ENI? (RVPN-01-701.)
3. Do the data ENIs carry IPv6 addresses, and are front ends ever deployed on a shared layer-2 segment? (RVPN-01-702, RVPN-01-710.)
4. What persists the exit configuration today (networkd, ifupdown, a unit running the script), and did any operator follow the printed advice to include `/etc/pfe-dsr.nft`? (RVPN-01-705, RVPN-01-708.)
5. Does any exit host run software that changes `net.ipv4.ip_forward` or enables IPv6 forwarding after `backend-dsr.sh` has run (container runtime, VPN bring-up, `sysctl --system` ordering)? (RVPN-01-706.)
6. Is `gre0` guaranteed down on every exit? (RVPN-01-705.)
7. Is FoU still required by any exit with a userspace decapsulator; if not, can it be removed and the default changed to GRE? (RVPN-01-713.)
8. Is `ethtool -L <iface> combined 1` applied by automation on every front end, or could a multi-queue NIC be bound? (RVPN-01-711.)
9. The generic XDP hook also takes DHCP replies for the data NIC's own address: has a front end been observed to lose its lease or its default route on the data NIC, which would break the gateway discovery at the next restart? (RVPN-01-710; `--gateway-mac` avoids the dependency.)
10. Would an authenticated tunnel (ESP with NAT traversal, or WireGuard) on the cloud-to-exit leg be acceptable operationally, and who would own key distribution? (RVPN-01-704.)
11. `modprobe tunnel4` (backend-dsr.sh:18, README:76) loads the IPIP demultiplexer rather than `ip_gre`; is this intentional, given that `ip link add type gre` loads `ip_gre` on demand?
12. Is a Linux lab (two network namespaces or a veth pair) available to the project for the dynamic tests listed under A34, which Red Team Ltd could not execute on macOS?

Linux-only confirmations still owed after ee2a0c6. The following remediated behaviours were verified by inspection or in isolation on the assessment host and need one run on a Linux lab (two network namespaces or a veth pair) to be closed as exercised:

- RVPN-01-702: a sustained stream of at least 5,000 non-forwarded frames with no transmission, after which IPv4 to a mapped address must still be forwarded and `fillEmpty` must not grow.
- RVPN-01-701 and RVPN-01-707 end to end: malformed header-length frames counted in `drop=` without a restart, and fragmented UDP and TCP delivered intact at a GRE endpoint.
- RVPN-01-705 and RVPN-01-706: `nft -f` of the generated ruleset on a current nftables, including acceptance of the `"pfe-gre*"` wildcard, the add-delete-recreate sequence as one transaction, the forward chain dropping inner IPv4 and IPv6 routed through a tunnel device, and the `gre0` fallback drop.
- RVPN-01-713: the unit under systemd with `kernel.unprivileged_bpf_disabled=2`, confirming that CAP_BPF suffices and that `ProtectKernelTunables=`, `PrivateDevices=` and `ProtectSystem=strict` do not interfere with the AF_XDP socket and the `/sys/class/net` and `/proc/net` reads.

- RVPN-01-712: `txInvalid=` staying at zero under load with the `repr(C)` descriptor, and the `pkt_slice` bound assertion never firing on kernel-produced descriptors.
- RVPN-01-710 and RVPN-01-709: the cache lookup against a live `/proc/net/arp` with an incomplete entry, and the probe path leaving the fill ring balanced.
- RVPN-01-703: the journal line at `-d` showing the /24 and at `-dd` the full address.